

@azdocuments

DIGITAL DESIGN AND COMPUTER ORGANIZATION

SUBJECT CODE: 1BCS303

MODULE-2

Name:

USN:

College:

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

DIGITAL DESIGN AND COMPUTER ORGANIZATION

1BCS303

MODULE-2

Combinational Logic: Introduction, Combinational Circuits, Analysis of Combinational Circuits, Design Procedure, Binary Adder – Subtractor, Decoders, Encoders, Multiplexers. Synchronous

Sequential Logic: Introduction, Sequential Circuits, Storage Elements: Latches, Flip-Flops.

Textbook-1: Chapter- 4 (4.1, 4.2, 4.3, 4.4, 4.5, 4.9, 4.10, 4.11), Chapter-5 (5.1, 5.2, 5.3, 5.4)

2 COMBINATIONAL LOGIC

2.1 INTRODUCTION

In digital systems, logic circuits are mainly classified into **combinational circuits** and **sequential circuits**. The most important difference between them is whether the **past history of the circuit affects its present output**.

A **combinational logic circuit** is a circuit in which the output at any instant depends **only on the present values of the inputs**. In other words, if we know the current input values, we can determine the current output without knowing what the inputs were earlier. A combinational circuit performs an operation that can be specified using a set of **Boolean functions**.

For example, suppose a circuit has inputs $A$and B, and its output is

$$F = A + B$$

Here, the output Fdepends only on the present values of $A$and B. If $A = 0$and $B = 1$, then

$$F = 0 + 1 = 1$$

It does not matter what values $A$and Bhad previously.

A **sequential circuit**, on the other hand, contains **storage elements** in addition to logic gates. Therefore, its output depends not only on the present inputs but also on the **current state of the storage elements**. Since the state of these storage elements depends on previous inputs, the output of a sequential circuit can depend on both the **present inputs and past inputs**.

We can express the basic distinction as:

Combinational: Output = function of present inputs

whereas

Sequential: Output = function of present inputs + previous state

Sequential circuits are used as building blocks in digital systems and are studied in later chapters.

2.2 COMBINATIONAL CIRCUITS

A **combinational circuit** consists of an interconnection of **logic gates**. The gates respond to the values of signals applied to their inputs and produce corresponding output signals. Thus, the circuit takes binary information from the input and transforms it into the required binary output.

Consider a combinational circuit having $n$input variables and $m$output variables.

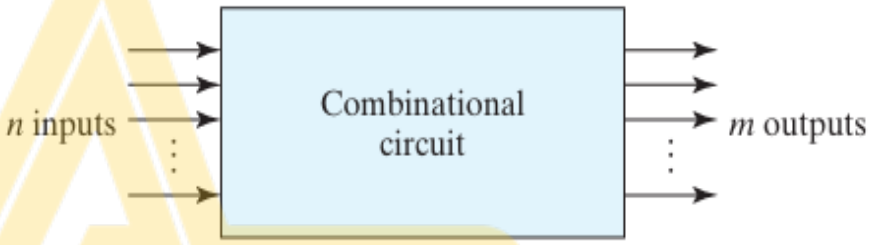


FIGURE 4.1
Block diagram of combinational circuit

The general structure can be understood as:

n inputs → Combinational Logic Circuit → m outputs

The $n$input binary variables come from some external source. These inputs enter the combinational logic circuit, where they are processed by logic gates. The resulting $m$output variables are then sent to an external destination.

Physically, an input or output variable is actually represented by an **analog electrical signal**, typically a voltage. However, in digital logic we interpret the voltage as one of two binary values:

Logic 0

or

Logic 1

Therefore, although the physical signal is analog, we treat it as a binary signal for digital logic operations. Logic simulators normally display only the binary values 0and 1, rather than the actual analog voltage waveform.

In many practical systems, the source and destination of these signals are **storage registers**. A very important point is that if storage registers are included together with the combinational gates, then the **complete circuit is considered a sequential circuit**, because the registers introduce storage and therefore state.

Number of Input Combinations

Suppose a combinational circuit has $n$binary input variables.

Since every input can have either 0or 1, the total number of possible input combinations is

$$\boxed{2^n}$$

For example, if there are two inputs $A$and B, then

$$2^2 = 4$$

possible combinations exist:

$$00,\ 01,\ 10,\ 11$$

If there are three inputs A, B, C, then

$$2^3 = 8$$

possible combinations exist.

For **each possible input combination**, there is exactly **one possible value for each output variable**. This property allows us to completely describe a combinational circuit using a **truth table**.

For example, if a circuit has three inputs, its truth table must contain

$$2^3 = 8$$

input combinations.

If the circuit has $m$outputs, then each row of the truth table specifies the value of all $m$outputs for that particular input combination.

Boolean Functions of a Combinational Circuit

A combinational circuit can also be described mathematically using **Boolean functions**.

If a circuit has $n$input variables and $m$output variables, then we can describe the circuit using mBoolean functions, one for each output.

For example, suppose a circuit has three inputs $A, B, C$and two outputs F_1, F_2. We could describe it as

$$F_1 = f_1(A, B, C)$$

and

$$F_2 = f_2(A, B, C)$$

Thus, each output is expressed in terms of the input variables.

The important idea is that **the Boolean equations completely describe the behavior of a combinational circuit**.

Standard Combinational Circuits

Many combinational circuits are used repeatedly in digital-system design. Because these circuits perform commonly required functions, they are available as **standard components**.

Important examples include:

Adders, Subtractors, Comparators, Decoders, Encoders, Multiplexers

These circuits can be available as **integrated circuits (ICs)** and are classified as **medium-scale integration (MSI)** circuits.

The same types of circuits are also used as **standard cells** inside more complex **very-large-scale integration (VLSI)** circuits, such as **application-specific integrated circuits (ASICs)**.

The important idea is that instead of designing every small function from individual gates every time, designers can use these standard combinational building blocks and connect them together to construct larger digital systems.

For example, an adder can be treated as a basic building block:

Binary Inputs → Adder → Binary Sum

Similarly, a multiplexer can be used as a standard block for selecting one of several input signals.

2.3 ANALYSIS OF COMBINATIONAL CIRCUITS

The **analysis of a combinational circuit** means determining **what function the circuit performs**. In other words, we start with a given logic-gate diagram and find out the logical operation implemented by that circuit.

The analysis normally ends with one or more of the following: a **Boolean function**, a **truth table**, or an explanation of what the circuit actually does. If the circuit diagram is already given a function name or a description of what it is supposed to accomplish, then the purpose of analysis is mainly to **verify whether the circuit really performs that specified function**.

Analysis can be performed manually by deriving the Boolean expressions or truth table. It can also be performed using a **computer simulation program**.

First Step: Check Whether the Circuit Is Combinational

Before analyzing the logic gates, the first thing we must verify is that the circuit is actually a **combinational circuit** and not a sequential circuit.

A combinational circuit contains logic gates but **does not contain memory elements or feedback paths**.

A **feedback path** occurs when the output of one gate is connected back to the input of another gate, and the output of that second gate eventually becomes part of the input to the first gate. Such feedback creates a dependency on the previous state of the circuit.

Therefore:

Combinational circuit → No feedback, no memory

Feedback or memory → Sequential circuit

Sequential circuits require special analysis methods because their outputs depend on stored state. Such circuits are not considered in this section.

Obtaining Boolean Functions from a Logic Diagram

Once we confirm that the circuit is combinational, we can determine its output Boolean functions.

The basic idea is very simple: **do not try to write the final output equation immediately**. Instead, move through the circuit step by step, giving meaningful names to intermediate gate outputs.

Step 1: Label the First-Level Gate Outputs

First, identify gates whose inputs contain at least one of the original circuit inputs. Give their outputs arbitrary but meaningful symbols such as

$$T_1, T_2, T_3$$

Then write the Boolean expression for each of these gate outputs.

Step 2: Move Toward the Output

Next, identify gates whose inputs consist of the original input variables and/or outputs that have already been labelled.

Again, assign symbols to their outputs and write their Boolean functions.

Step 3: Continue the Process

Repeat the same process until we reach the final outputs of the circuit.

Step 4: Substitute

Finally, substitute the intermediate expressions into the output equations repeatedly until the final outputs are written **only in terms of the original input variables**.

Thus, the overall procedure is:

Logic diagram → Intermediate equations → Substitution → Final Boolean functions

Example: Analysis of the Circuit in Fig. 4.2

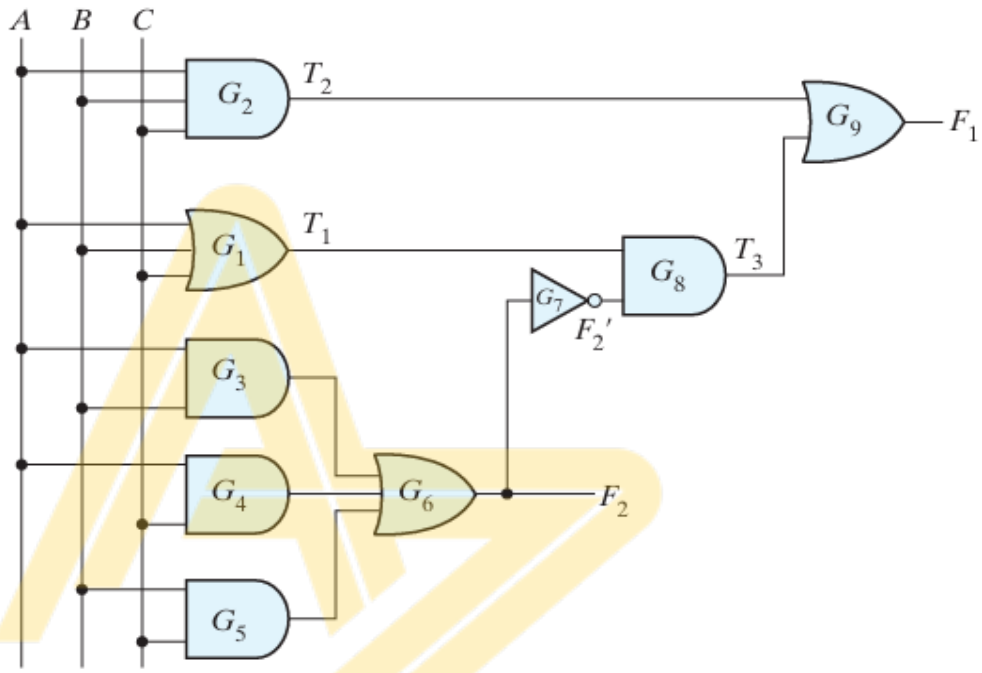


FIGURE 4.2
Logic diagram for analysis example

The circuit has three binary inputs:

$$A,\ B,\ C$$

and two binary outputs:

$$F_1,\ F_2$$

The outputs of several intermediate gates are represented by T_1, T_2, T_3.

The first outputs that can be directly determined from the circuit inputs are F_2, T_1,and T_2.

From the gates in the diagram:

$$\boxed{F_2 = AB + AC + BC}$$

The output T_1 is obtained by an OR operation on the three inputs:

$$\boxed{T_1 = A + B + C}$$

The output T_2 is obtained by an AND operation:

$$\boxed{T_2 = ABC}$$

Now we move to the next level of the circuit.

The next intermediate output is

$$\boxed{T_3 = F_2'T_1}$$

and the final output F_1 is

$$\boxed{F_1 = T_3 + T_2}$$

Now we have equations for the intermediate signals, but F_1 is not yet expressed completely in terms of A, B, C. Therefore, we substitute.

Deriving F_1

Starting with

$$F_1 = T_3 + T_2$$

substitute

$$T_3 = F_2'T_1$$

and

$$T_2 = ABC$$

Therefore,

$$F_1 = F_2'T_1 + ABC$$

Now substitute

$$F_2 = AB + AC + BC$$

and

$$T_1 = A + B + C$$

to obtain

$$F_1 = (AB + AC + BC)'(A + B + C) + ABC$$

Using DeMorgan's theorem,

$$(AB + AC + BC)' = (AB)'(AC)'(BC)'$$

and

$$(AB)' = A' + B'$$
$$(AC)' = A' + C'$$
$$(BC)' = B' + C'$$

Therefore,

$$F_1 = (A' + B')(A' + C')(B' + C')(A + B + C) + ABC$$

After Boolean simplification, this becomes

$$\boxed{F_1 = A'BC' + A'B'C + AB'C' + ABC}$$

Therefore, the two output functions of the circuit are

$$\boxed{F_1 = A'BC' + A'B'C + AB'C' + ABC}$$

and

$$\boxed{F_2 = AB + AC + BC}$$

The important point is that we obtained these equations **systematically by moving from the input side toward the output side**.

What Does This Circuit Actually Do?

Simply obtaining a Boolean expression does not always immediately tell us what a circuit is doing.

In this example, if we examine the Boolean functions and construct the corresponding truth table, we find that the behavior matches the truth table of a **full-adder**.

The correspondence is:

$$A, B, C \leftrightarrow x, y, z$$
$$F_1 \leftrightarrow S$$
$$F_2 \leftrightarrow C$$

where Srepresents the **sum** and Crepresents the **carry**.

So the circuit performs the operation of a **full-adder**.

This is an important lesson in circuit analysis: the Boolean equations tell us exactly how the circuit behaves mathematically, while the truth table can help us recognize a familiar digital circuit.

Obtaining the Truth Table Directly

We can also analyze the circuit by constructing its truth table directly, without first deriving the complete Boolean expressions.

The first step is to determine the number of input variables. Here there are three inputs:

$$A, B, C$$

Therefore, the number of possible combinations is

$$2^3 = 8$$

So we list all eight combinations:

$$000,\ 001,\ 010,\ 011,\ 100,\ 101,\ 110,\ 111$$

Then we calculate the outputs of the intermediate gates one level at a time.

The procedure is:

Inputs → First-level gate outputs → Next-level outputs → Final outputs

For the first-level outputs,

$$T_1 = A + B + C$$

so T_1 is 1whenever **at least one** of A, B, C is 1.

Similarly,

$$T_2 = ABC$$

so T_2 is 1only when

$$A = B = C = 1$$

For

$$F_2 = AB + AC + BC$$

the output is 1whenever **at least two of the three inputs are 1**.

For example, if

$$A = 1,\ B = 1,\ C = 0$$

then

$$F_2 = (1)(1) + (1)(0) + (1)(0)$$
$$F_2 = 1 + 0 + 0 = 1$$

But if

$$A = 1,\ B = 0,\ C = 0$$

then

$$F_2 = (1)(0) + (1)(0) + (0)(0) = 0$$

Thus F_2represents the **carry function** of a full-adder.

Next,

$$T_3 = F_2' T_1$$

Therefore $T_3 = 1$only when both $F_2'$and $T_1$are 1.

Finally,

$$F_1 = T_3 + T_2$$

so F_1becomes 1whenever either T_3or T_2, or both, are 1.

The resulting truth table is:

A	**B**	**C**	F_2	F_2'	T_1	T_2	T_3	F_1
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

Notice that $F_1$is 1for inputs having an **odd number of 1s**, while $F_2$is 1when **two or three inputs are 1**. This is exactly the behavior associated with the sum and carry outputs of a full-adder.

Computer Simulation

Another method of analyzing a combinational circuit is **logic simulation**.

A simulation program can apply different input combinations to the circuit and observe the resulting outputs. This is particularly useful when we want to verify that an implemented circuit actually matches its required specification.

However, simulation is not always the most practical method for analysis because a circuit with many input variables can have a very large number of possible input combinations.

For $n$inputs:

$$\boxed{2^n \text{ input combinations}}$$

For example:

$$n = 3 \Rightarrow 2^3 = 8$$

but

$$n = 10 \Rightarrow 2^{10} = 1024$$

and

$$n = 20 \Rightarrow 2^{20} = 1{,}048{,}576$$

Therefore, as the number of inputs increases, testing every possible combination becomes increasingly difficult.

Simulation is especially useful for **verification**, and it generally requires an **HDL model** of the circuit.

Practice Exercise 4.1

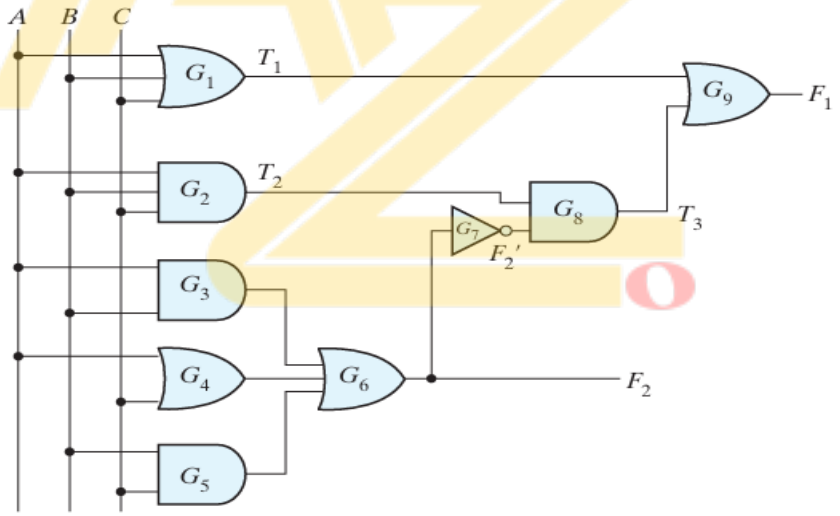


FIGURE PE4.1

Answer: $T_1 = A + B + C$
$T_2 = ABC$
$F_2 = AB + A + B + BC = A + B + BC = A + B$
$F_2' = A'B'$
$T_3 = (ABC)(A'B') = 0$
$F_1 = T1 = A + B + C$

The given circuit has three inputs:

$$A,\ B,\ C$$

and outputs F_1 and F_2.

We begin by identifying the intermediate signals.

The first signal is

$$\boxed{T_1 = A + B + C}$$

The second is

$$\boxed{T_2 = ABC}$$

The circuit gives

$$F_2 = AB + A + B + BC$$

Now simplify it.

Since

$$A + AB = A$$

and

$$B + BC = B$$

we get

$$F_2 = A + B$$

Therefore,

$$\boxed{F_2 = A + B}$$

Its complement is

$$\boxed{F_2' = A'B'}$$

The next intermediate output is

$$T_3 = (ABC)(A'B')$$

Rearranging,

$$T_3 = AA'BB'C$$

Since

$$AA' = 0$$

we get

$$\boxed{T_3 = 0}$$

Finally, from the circuit,

$$F_1 = T_1 + T_3$$

Since $T_3 = 0$,

$$F_1 = T_1 + 0$$

Therefore,

$$\boxed{F_1 = T_1 = A + B + C}$$

Hence the final answers are

$$\boxed{F_1 = A + B + C}$$

and

$$\boxed{F_2 = A + B}$$

2.4 DESIGN PROCEDURE

In analysis, we start with a **logic circuit** and determine what it does. In design, we start with a **required operation or specification** and construct a logic circuit that performs that operation.

Thus, the overall idea is:

$$\boxed{\text{Specification} \rightarrow \text{Truth Table} \rightarrow \text{Boolean Functions} \rightarrow \text{Logic Circuit}}$$

The design process ultimately produces either a **logic circuit diagram** or a set of **Boolean functions** from which the circuit diagram can be constructed.

Steps in the Design of a Combinational Circuit

The design procedure consists of four main steps.

Step 1: Determine Inputs and Outputs

First, we study the specification of the required circuit and determine:

- how many **input variables** are required,
- how many **output variables** are required, and
- what symbols will represent each input and output.

For example, suppose a circuit requires three inputs. We may assign them as

$$A,\ B,\ C$$

If it has two outputs, we may call them

$$F_1,\ F_2$$

So the first stage is:

$$\boxed{\text{Specification} \rightarrow \text{Inputs and Outputs}}$$

Step 2: Construct the Truth Table

Once the inputs and outputs have been identified, the next step is to construct the **truth table**.

If there are $n$input variables, there will be

$$\boxed{2^n}$$

possible combinations of inputs.

For example, with three inputs,

$$2^3 = 8$$

so the truth table will contain eight possible input combinations.

The input portion of the truth table is obtained by listing the binary numbers from

$$0 \text{ to } 2^n - 1$$

The output values are then determined from the **given specification** of the circuit.

This step is extremely important because the truth table provides the exact relationship between the inputs and outputs.

A wrong interpretation of the verbal specification will result in a wrong truth table, and therefore the final circuit will also be wrong.

So:

$$\boxed{\text{Correct specification} \rightarrow \text{Correct truth table}}$$

$$\boxed{\text{Wrong interpretation} \rightarrow \text{Wrong circuit}}$$

Step 3: Obtain Simplified Boolean Functions

After constructing the truth table, we derive a Boolean function for each output.

For example, if the circuit has two outputs $F_1$and F_2, we derive

$$F_1 = f_1(A, B, C)$$

and

$$F_2 = f_2(A, B, C)$$

These Boolean functions are then **simplified** using an appropriate method.

Several methods can be used, including:

Boolean algebra

Karnaugh map / Map method

and

Computer-based simplification

For example, suppose we obtain

$$F = AB + AB'$$

Using Boolean algebra,

$$F = A(B + B')$$

Since

$$B + B' = 1$$

we get

$$F = A$$

Thus, the simplified function is

$$\boxed{F = A}$$

A simpler Boolean expression generally leads to a simpler circuit.

Step 4: Draw and Verify the Logic Diagram

Once the Boolean functions have been simplified, we use them to construct the **logic diagram**.

For example, if

$$F = AB + CD$$

we can implement it using two AND gates followed by an OR gate.

After constructing the circuit, we must **verify that it performs the required operation**.

Verification can be performed manually using Boolean analysis or a truth table, or it can be performed using a simulation program.

Therefore, the complete design procedure is:

Truth Table in Circuit Design

A truth table for a combinational circuit contains two main parts: **input columns** and **output columns**.

If there are $n$inputs, the input columns contain the 2^npossible combinations.

For example, for four inputs:

$$A, B, C, D$$

the number of combinations is

$$2^4 = 16$$

Therefore, the truth table has 16 input combinations.

The output values are determined according to the required behavior described in the circuit specification.

The output functions represented by the truth table provide the **exact definition of the combinational circuit**.

This is why the truth table is one of the most important stages of circuit design.

Choosing the Best Boolean Expression

An important point is that there may be **more than one simplified Boolean expression** for the same function.

For example, the same logical function may sometimes be represented in different simplified forms depending on the simplification method or the requirements of the circuit.

Therefore, simply finding a mathematically simplified expression is not always enough. The designer must choose an implementation based on the practical requirements of the application.

Some important factors include the **number of gates** required. Fewer gates generally mean a smaller and more economical circuit.

The **number of inputs to each gate** is also important. A circuit may be logically correct but impractical if it requires gates with an excessive number of inputs.

Another factor is the **propagation time** of signals through the gates. Each gate introduces some delay, so a circuit with many levels of gates may take longer to produce its final output.

The **number of interconnections** is also important because excessive wiring makes a circuit more complicated and can increase implementation difficulty.

Another consideration is the **driving capability** of a gate. The output of one gate can drive only a limited number of other gates. This limitation is sometimes described in terms of the number of gates that the output can be connected to.

Therefore, practical circuit design must consider:

- Number of gates
- Number of gate inputs
- Propagation delay
- Number of interconnections
- Driving capability

along with other application-specific requirements.

There is therefore no single rule that always tells us which implementation is best. The importance of each constraint depends on the particular application.

Usually, simplification begins with an elementary objective, such as obtaining a Boolean function in a standard form. After that, further optimization can be performed to satisfy other performance requirements.

Code Conversion

One of the important applications of combinational-circuit design is **code conversion**.

Different digital systems may use different binary codes to represent the same information. For example, one system may use **BCD**, while another may use **Excess-3**.

If the output of one system must be connected to another system using a different code, a **code converter** is required.

A code converter is therefore a combinational circuit that converts one binary code into another.

The basic idea is:

Code A → Code Converter → Code B

The code converter makes two systems compatible even though they use different binary representations for the same information.

Table 4.2
Truth Table for Code Conversion Example

Input BCD				Output Excess-3 Code			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

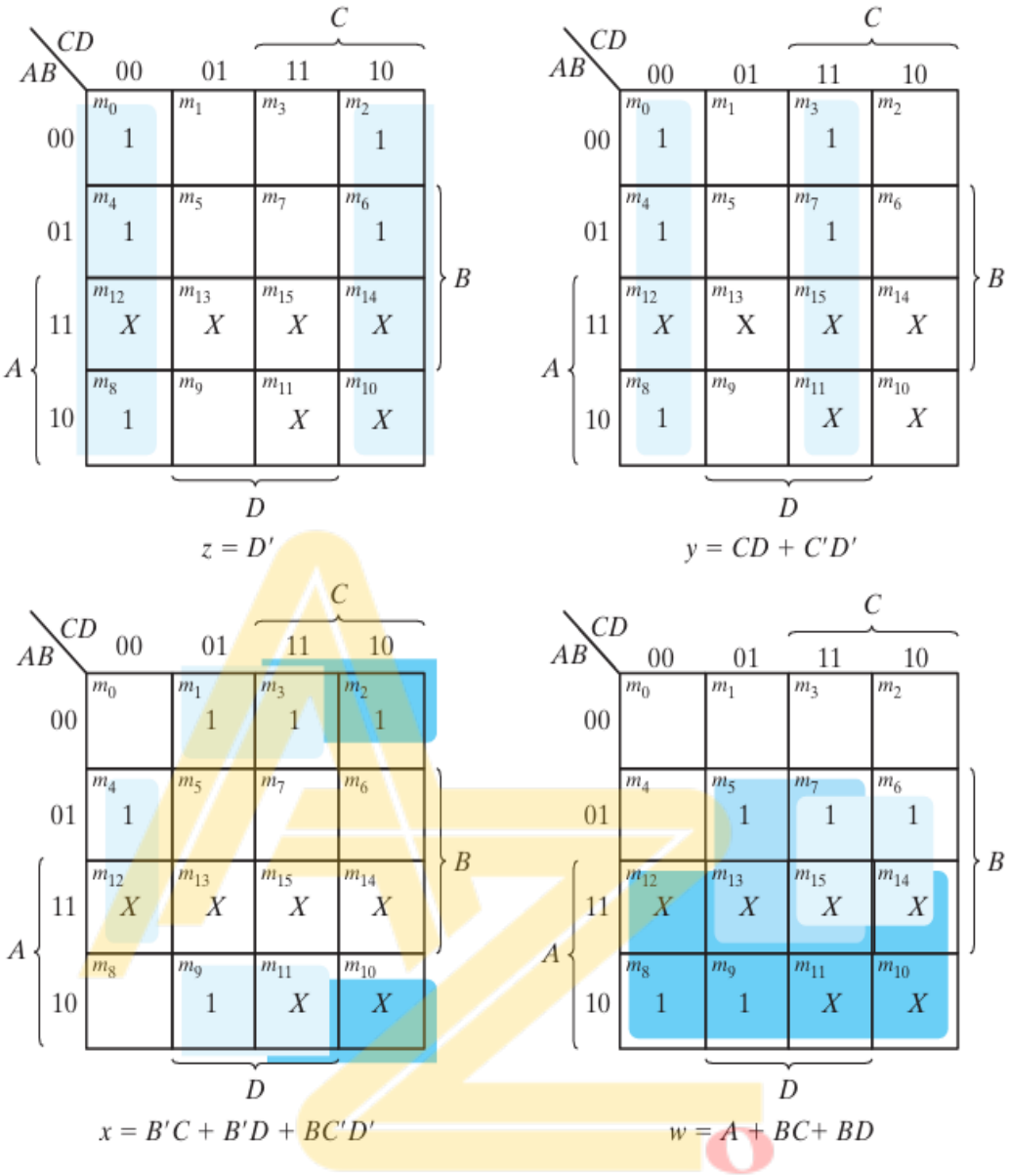


FIGURE 4.3
Maps for BCD-to-excess-3 code converter

BCD-to-Excess-3 Code Converter

The textbook illustrates the design procedure using a circuit that converts **BCD to Excess-3 code**.

BCD uses four bits to represent each decimal digit. Therefore, the input to the converter requires four binary variables:

$$\boxed{A, B, C, D}$$

The converter also produces four output variables:

$$\boxed{w, x, y, z}$$

Therefore:

4 inputs and 4 outputs

The required relationship between the BCD inputs and Excess-3 outputs is represented by the truth table.

$$z = D'$$

$$y = CD + C'D' = CD + (C + D)'$$

$$x = B'C + B'D + BC'D' = B'(C + D) + BC'D'$$

$$= B'(C + D) + B(C + D)'$$

$$w = A + BC + BD = A + B(C + D)$$

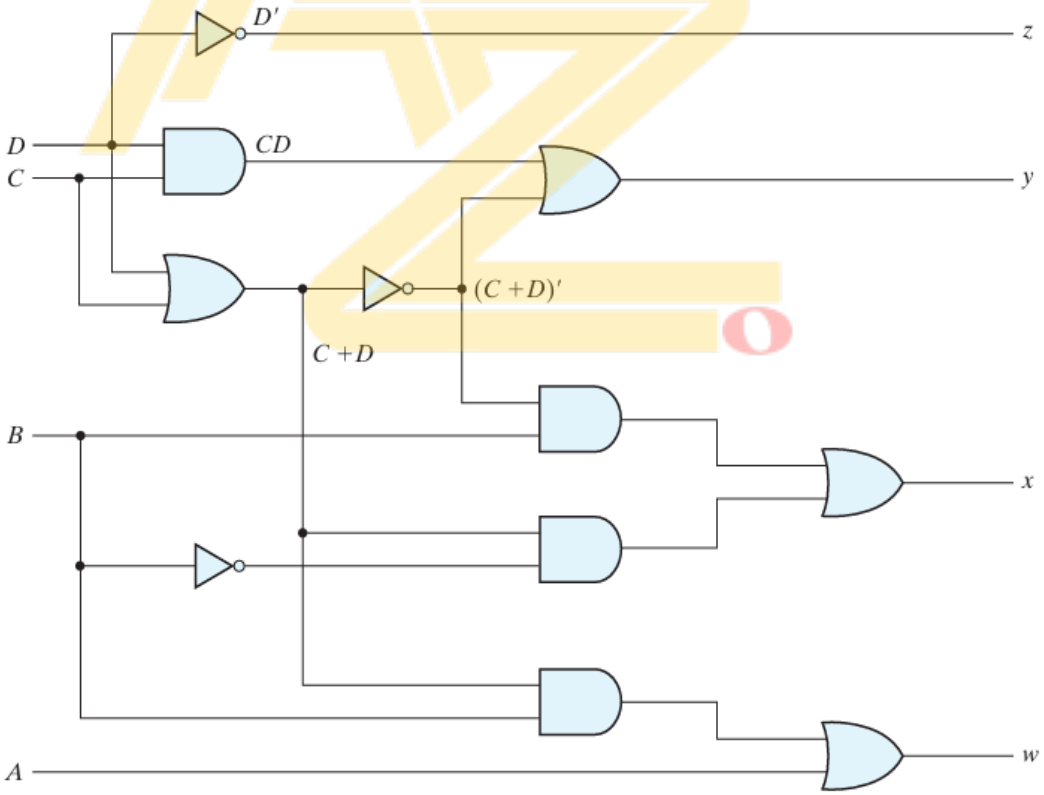


FIGURE 4.4
Logic diagram for BCD-to-excess-3 code converter

Why Are There Only 10 Valid Rows?

Four binary variables can theoretically produce

$$2^4 = 16$$

possible combinations.

These combinations range from

$$0000$$

to

$$1111$$

However, BCD is used to represent decimal digits 0through 9, so only **10 combinations** are valid.

Thus, the converter's truth table contains only:

$$\boxed{10 \text{ valid BCD combinations}}$$

The remaining six combinations correspond to binary values 10through 15, which have no meaning in standard BCD representation.

These six combinations are treated as **don't-care combinations**.

They are usually represented by

$$\boxed{X}$$

in a Karnaugh map.

Don't-Care Conditions

A don't-care combination is an input combination for which the output does not matter because that input condition is assumed never to occur during normal operation.

Here, the invalid BCD inputs are:

$$1010, 1011, 1100, 1101, 1110, 1111$$

corresponding to decimal values

$$10,11,12,13,14,15$$

Since these values are not valid BCD representations of decimal digits, the circuit does not need to produce a particular output for them.

Therefore, during Boolean simplification, we can assign the output for these combinations as either

$$0$$

or

$$1$$

depending on which choice produces the **simpler circuit**.

This is a very important concept in Karnaugh-map simplification:

$$\boxed{\text{Don't-care } (X) \text{ can be treated as 0 or 1}}$$

but only when doing so helps simplify the circuit.

Excess-3 Code

The Excess-3 code is obtained by adding 3to the binary value corresponding to the decimal digit.

For example, for decimal 2:

$$2 + 3 = 5$$

The binary representation of 5is

$$0101$$

Therefore,

$$\boxed{\text{Excess-3 code for decimal } 2 = 0101}$$

Similarly, the truth table gives the corresponding Excess-3 output for every valid BCD input.

Simplifying the Four Outputs

Since there are four outputs,

$$w,\ x,\ y,\ z$$

we need to simplify **four Boolean functions**.

Therefore, four Karnaugh maps are used.

Each map represents one output function in terms of the four inputs:

$$A, B, C, D$$

The 1's in each map are obtained by examining the corresponding output column of the truth table.

For example, if the $z$output column contains five 1's, then the Karnaugh map for $z$will contain five 1's in the cells corresponding to those input minterms.

The six invalid BCD combinations are marked as

$$X$$

and can be used during grouping to obtain larger groups and hence simpler expressions.

The resulting simplified sum-of-products expressions are:

$$\boxed{z = D'}$$
$$\boxed{y = CD + C'D'}$$
$$\boxed{x = B'C + B'D + BC'D'}$$

and

$$\boxed{w = A + BC + BD}$$

These expressions directly describe the four outputs of the BCD-to-Excess-3 converter.

Sharing Common Subcircuits

An important design idea appears when these expressions are examined carefully.

The expressions can be rearranged algebraically:

For z:

$$\boxed{z = D'}$$

For y:

$$y = CD + C'D'$$

Using the identity

$$C'D' = (C + D)'$$

we can write

$$\boxed{y = CD + (C + D)'}$$

For x:

Starting with

$$x = B'C + B'D + BC'D'$$

factor B' from the first two terms:

$$x = B'(C + D) + BC'D'$$

Now note that

$$(C + D)' = C'D'$$

Therefore,

$$x = B'(C + D) + B(C + D)'$$

so

$$\boxed{x = B'(C + D) + B(C + D)'}$$

For w:

Starting with

$$w = A + BC + BD$$

factor B:

$$w = A + B(C + D)$$

Therefore,

$$\boxed{w = A + B(C + D)}$$

The Important Common Subcircuit: $C + D$

Now notice something very useful.

The expression

$$C + D$$

appears in three different outputs:

$$y = CD + (C + D)'$$
$$x = B'(C + D) + B(C + D)'$$
$$w = A + B(C + D)$$

Therefore, instead of separately constructing $C + D$three times, we can construct it **once** and use its output in all three circuits.

This is the main idea behind **multilevel logic optimization**.

The common subcircuit is:

$$\boxed{T = C + D}$$

Then the outputs can use T:

$$w = A + BT$$
$$x = B'T + BT'$$

and $y$can use T'together with CD:

$$y = CD + T'$$

This reduces the amount of hardware required.

Why Multilevel Logic Can Be Better

If we implement the original sum-of-products expressions independently, the implementation requires, excluding input inverters:

$$\boxed{\text{7 AND gates and 3 OR gates}}$$

The optimized multilevel implementation requires:

$$\boxed{\text{4 AND gates, 4 OR gates, and 1 inverter}}$$

Although the number of OR gates increases, the total implementation becomes more economical because the common subcircuit $C + D$is shared among multiple outputs.

If only the normal, non-complemented inputs are available, the required input inverters are also reduced.

For the first implementation, inverters are required for:

$$B, C, D$$

whereas the second implementation requires inverters for:

$$B, D$$

Thus, the three-level implementation uses fewer gates, and the gates require no more than two inputs.

The Main Concept: Subcircuit Sharing

The most important design idea demonstrated by this example is that **a common portion of logic can be shared by several outputs**.

Here,

$$\boxed{C + D}$$

is used in the formation of

$$x, y, w$$

Instead of building $C + D$separately for each output, it is calculated once and reused.

This is particularly useful in **multiple-output combinational circuits**.

Therefore:

Multiple outputs → Find common subcircuits → Share them → Reduce hardware

Modern logic-synthesis tools automatically search for such common subcircuits and exploit them to produce more efficient implementations.

2.5 BINARY ADDER–SUBTRACTOR

Digital computers perform many information-processing tasks, and one of the most important tasks is performing **arithmetic operations**. Among all arithmetic operations, the most basic one is the addition of two binary digits. Binary addition has four possible elementary operations:

$$0 + 0 = 0$$
$$0 + 1 = 1$$
$$1 + 0 = 1$$
$$1 + 1 = 10$$

The first three operations produce a one-bit sum. However, when both bits are 1, the result is 10. In this result, the rightmost bit is the sum bit and the leftmost bit is the **carry bit**. The carry is the higher-significant bit of the result. When binary numbers contain more than one bit, this carry must be added to the next higher-order pair of bits.

A combinational circuit that adds **two binary bits** is called a **half adder**. A combinational circuit that adds **three binary bits**, consisting of two significant data bits and a carry from the previous position, is called a **full adder**. The full adder can be constructed using two half adders.

A **binary adder–subtractor** is a combinational circuit that performs both **addition and subtraction of binary numbers**. The circuit is developed hierarchically. First, the half adder is designed. From the half adder, the full adder is developed. Then $n$full adders can be connected in cascade to construct an adder for two n-bit numbers. Subtraction is incorporated by using a complementing circuit.

2.5.1 Half Adder

A **half adder** is a combinational circuit that adds two binary bits. It has **two binary inputs and two binary outputs**. Let the two inputs be $x$and y. The two outputs are:

$$S = \text{Sum}$$
$$C = \text{Carry}$$

The carry Cbecomes 1 only when **both inputs are 1**. The sum Srepresents the **least significant bit** of the result. The carry C, when generated, represents the most significant bit of the two-bit result.

Half-Adder Truth Table

$x\ y\ C\ S$

0 0 0 0

$x\ y\ C\ S$

0 1 0 1

1 0 0 1

1 1 1 0

From the truth table, the simplified sum-of-products expressions can be obtained.

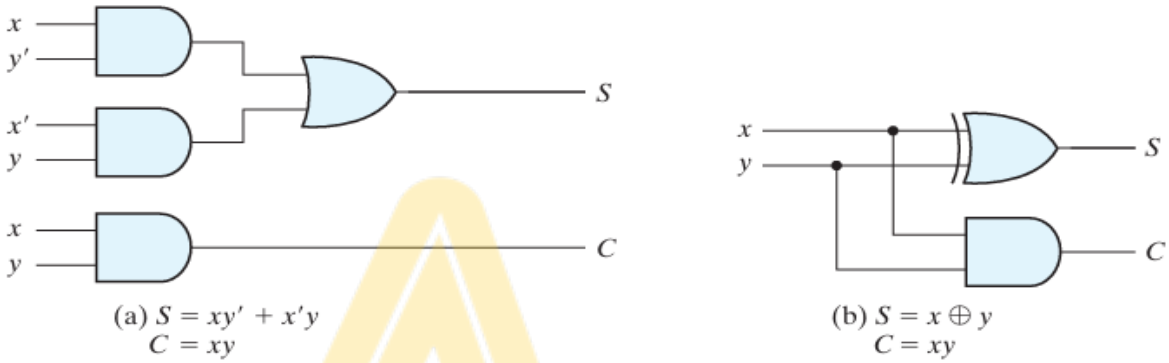


FIGURE 4.5
Implementation of half adder

The sum $S$is 1 when the two inputs are different. Therefore,

$$\boxed{S = x'y + xy'}$$

The carry $C$is 1 only when both $x$and $y$are 1. Therefore,

$$\boxed{C = xy}$$

Thus, the half adder can be implemented using an XOR gate and an AND gate:

$$\boxed{S = x \oplus y}$$
$$\boxed{C = xy}$$

The XOR-and-AND implementation is particularly useful because it is used to construct the full adder from two half adders.

2.5.2 FULL ADDER

When we add binary numbers containing several bits, the addition is performed **bit by bit from right to left**, beginning with the least significant bit. At the least significant position, there is initially no previous carry. However, at every subsequent position, we must add not only the two corresponding bits but also the carry generated by the previous position.

Therefore, a circuit that can add three bits is required.

A **full adder** is a combinational circuit that performs the arithmetic sum of **three binary bits**. It has three inputs and two outputs. The inputs are represented by

$$x, y, z$$

where $x$and $y$are the two significant bits being added, and $z$is the **carry-in** received from the previous lower-order position.

The outputs are

$$S = \text{Sum}$$

and

$$C = \text{Carry}$$

Three binary bits can have a total value from 0 to 3 in decimal. Since the binary representations of 2 and 3 require two bits, the circuit requires two outputs. The Soutput represents the least significant bit, while Crepresents the carry generated by the addition.

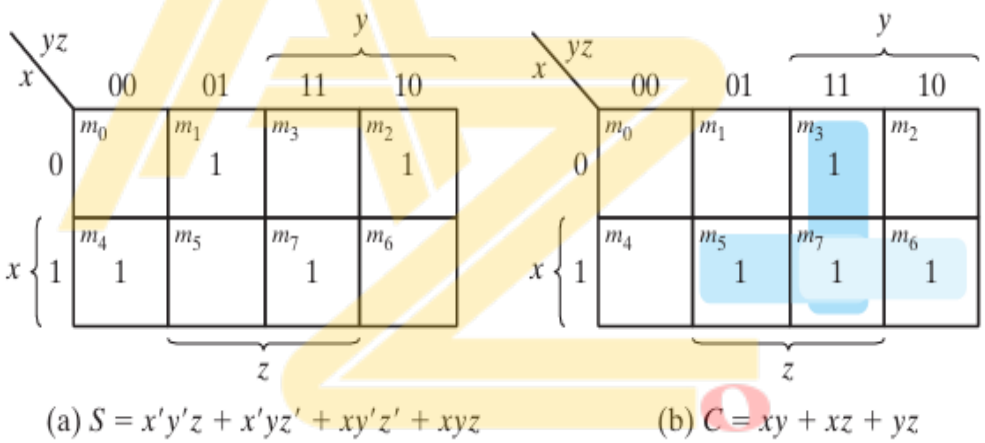


FIGURE 4.6
K-Maps for full adder

Full-Adder Truth Table

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1

x	y	z	C	S
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

There are eight possible combinations because there are three inputs:

$$2^3 = 8$$

The Soutput is 1 when **only one input is 1 or all three inputs are 1**. The carry Cbecomes 1 when **two or three inputs are 1**.

2.5.3 BOOLEAN EXPRESSIONS OF THE FULL ADDER

The same binary signals in a full adder can be interpreted in two different ways. Physically, the input signals represent binary digits that are being added arithmetically. When we construct the truth table or logic circuit, however, these same signals are treated as Boolean variables. The K-map can therefore be used to simplify the Boolean functions for $S$and C.

From the K-map, the simplified expression for the sum is

$$\boxed{S = x'y'z + x'yz' + xy'z' + xyz}$$

The simplified expression for the carry is

$$\boxed{C = xy + xz + yz}$$

The full adder can therefore be directly implemented in **sum-of-products form** using AND, OR, and NOT gates.

However, there is another important implementation: a full adder can be constructed using **two half adders and one OR gate**.

2.5.4 Full Adder Using Two Half Adders

The first half adder adds $x$and y. Its sum output is

$$x \oplus y$$

This result is then added to the carry-in $z$by the second half adder. Therefore, the final sum is

$$\boxed{S = z \oplus (x \oplus y)}$$

Since XOR is associative, this can also be viewed as the XOR of all three inputs.

Expanding the expression:

$$S = z \oplus (x \oplus y)$$

gives

$$S = z'(xy' + x'y) + z(xy' + x'y)'$$

Using DeMorgan's theorem,

$$(xy' + x'y)' = xy + x'y'$$

Therefore,

$$S = z'(xy' + x'y) + z(xy + x'y')$$

Expanding,

$$\boxed{S = xy'z' + x'yz' + xyz + x'y'z}$$

which is the same full-adder sum expression.

The carry generated by the two half adders is ORed together. Therefore,

$$C = z(xy' + x'y) + xy$$

Expanding,

$$C = xy'z + x'yz + xy$$

which simplifies to

$$\boxed{C = xy + xz + yz}$$

Thus, the full adder can be constructed using **two half adders and one OR gate**.

Practice Exercise 4.2

Explain how a half adder and a full adder differ in their functionality.

A **half adder** adds only two data bits and produces a sum and carry-out bit.

A **full adder** adds three input bits, consisting of two data bits and a carry-in bit, and produces a sum and carry-out bit.

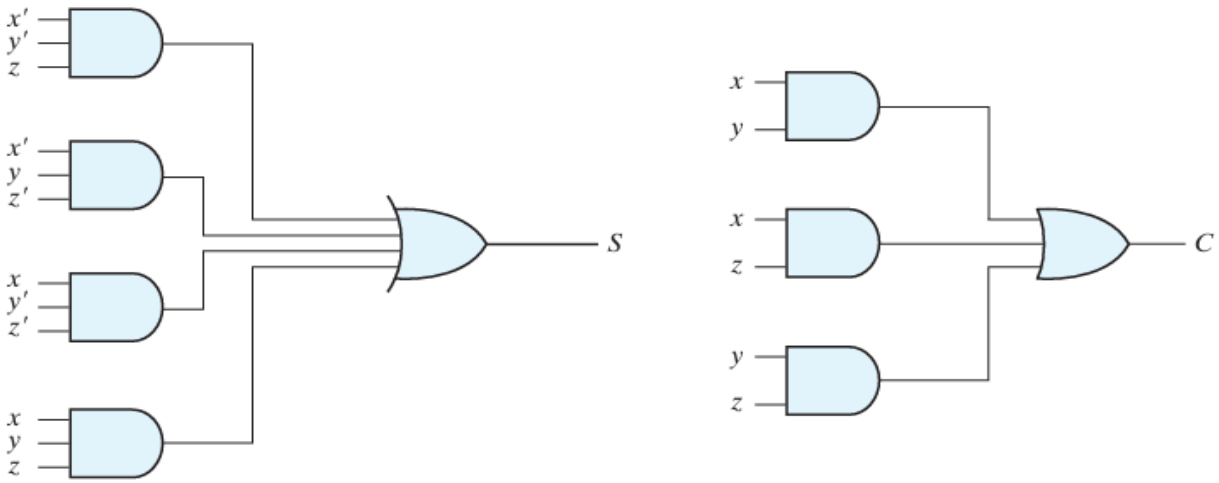


FIGURE 4.7
Implementation of full adder in sum-of-products form

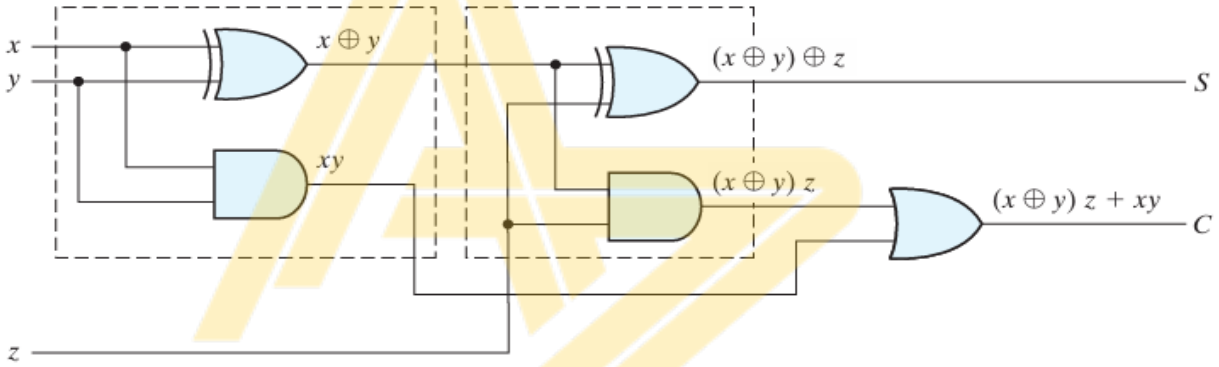


FIGURE 4.8
Implementation of full adder with two half adders and an OR gate

In short,

Half Adder: 2 inputs

Full Adder: 3 inputs including carry-in

2.5.5 BINARY ADDER

A **binary adder** is a digital circuit that produces the arithmetic sum of two binary numbers. It is constructed by connecting full adders in **cascade**. The carry output of one full adder is connected to the carry input of the next higher-order full adder.

To add two n-bit numbers, we can use either

n full adders

or

$$\boxed{1 \text{ half adder} + (n-1) \text{ full adders}}$$

When $n$full adders are used, the input carry to the least significant position is fixed at

$$\boxed{C_0 = 0}$$

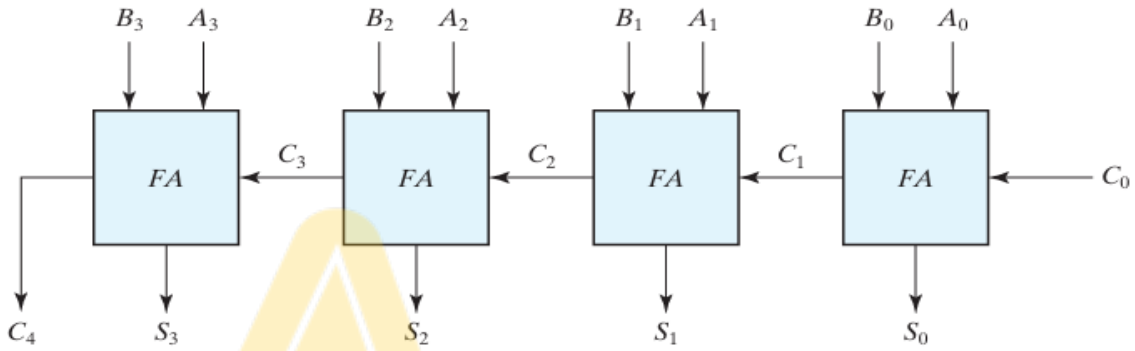


FIGURE 4.9
Four-bit adder

A four-bit binary adder consists of four full adders connected together. The bits of $A$and $B$are numbered from right to left, with subscript 0 representing the least significant bit:

$$A_3, A_2, A_1, A_0$$
$$B_3, B_2, B_1, B_0$$

The carry begins at C_0, passes through the full adders, and finally becomes C_4. The sum outputs are

$$S_3, S_2, S_1, S_0$$

This arrangement is known as a **four-bit ripple-carry adder** because the carry ripples from one full adder to the next.

Example: Addition Using a Four-Bit Adder

Consider

$$A = 1011$$

and

$$B = 0011$$

Their sum is

$$\boxed{S = 1110}$$

The addition is performed from the least significant position, $i = 0$, toward the most significant position.

The operation can be represented as:

Subscript i	3	2	1	0
C_i	0	1	1	0
A_i	1	0	1	1
B_i	0	0	1	1
S_i	1	1	1	0
C_{i+1}	0	0	1	1

The first addition is at the rightmost position:

$$1 + 1 + C_0$$

Since

$$C_0 = 0$$

we get

$$1 + 1 + 0 = 10$$

Thus,

$$S_0 = 0,\ C_1 = 1$$

The next full adder receives $C_1 = 1$as its carry-in. This process continues toward the left. The output carry $C_{i+1}$from one stage becomes the input carry $C_i$of the next higher-order stage. Therefore, the sum bits are generated from right to left as the carries become available.

For a four-bit adder, a direct classical truth-table design would require

$$2^9 = 512$$

entries because there are nine inputs. Instead, the iterative approach of cascading standard full-adder components provides a much simpler implementation.

2.5.6 CARRY PROPAGATION

When two binary numbers are added in parallel, all the bits of the augend and addend are available simultaneously. However, the outputs of a combinational circuit do not become correct immediately. The signals must propagate through the gates before the final output is obtained.

In a ripple-carry adder, the **longest propagation delay is caused by the carry**. Each sum bit depends on the carry entering its stage. Therefore, a particular sum output cannot reach its final value until the corresponding carry has reached that stage.

For example, consider S_3in a four-bit adder. The inputs $A_3$and $B_3$are available as soon as the input signals are applied. However, S_3also depends on C_3. The carry C_3cannot become final until $C_2$is available. Similarly, C_2depends on C_1, and C_1depends on C_0.

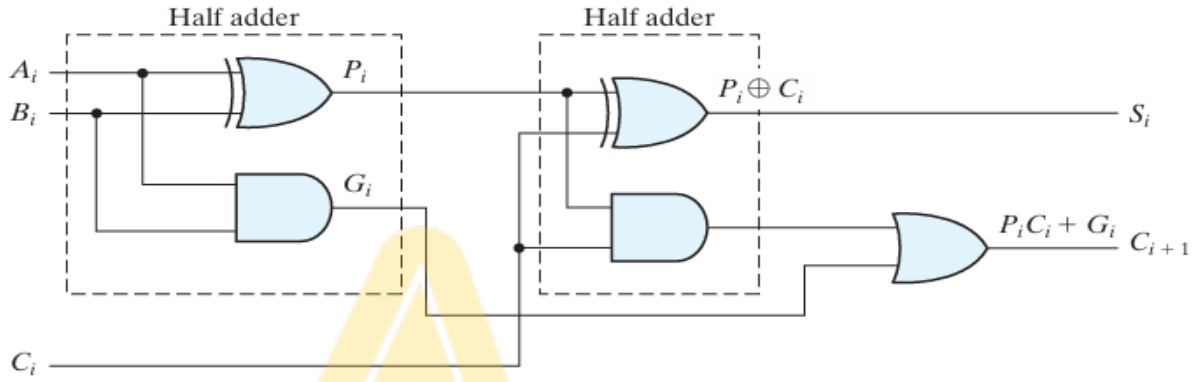


FIGURE 4.10
Full adder with *P* and *G* shown

Therefore,

$$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4$$

Only after the carry has propagated through all stages will the final sum and carry outputs become correct.

In a full adder, the carry signal passes through an AND gate and an OR gate, giving two gate levels. Therefore, for a four-bit adder:

$$2 \times 4 = 8$$

gate levels are required for the carry to propagate from $C_0$to C_4.

For an n-bit adder:

$$\boxed{2n \text{ gate levels}}$$

are required for carry propagation.

Carry propagation time is extremely important because it limits the speed of binary addition. Since other arithmetic operations are often implemented through successive additions, the time required for addition becomes critical. One way to reduce the delay is to use faster gates. However, physical circuits have limitations. Another approach is to use additional hardware to reduce the carry delay. The most widely used method is **carry lookahead logic**.

2.5.7 CARRY LOOKAHEAD ADDER

The purpose of a **carry lookahead adder** is to generate carry signals faster instead of allowing them to ripple sequentially from one stage to another.

For each bit position i, two new Boolean variables are defined:

$$\boxed{P_i = A_i \oplus B_i}$$

and

$$\boxed{G_i = A_i B_i}$$

Here,

$$G_i = \text{Carry Generate}$$

and

$$P_i = \text{Carry Propagate}$$

The carry-generate term $G_i$becomes 1 when both $A_i$and $B_i$are 1. In this case, that stage generates a carry regardless of the input carry C_i.

The carry-propagate term $P_i$determines whether an incoming carry will propagate through that stage to the next stage.

The sum output is

$$\boxed{S_i = P_i \oplus C_i}$$

and the carry output is

$$\boxed{C_{i+1} = G_i + P_i C_i}$$

Starting with

$$\boxed{C_0 = \text{input carry}}$$

we obtain

$$\boxed{C_1 = G_0 + P_0 C_0}$$

For the next stage,

$$C_2 = G_1 + P_1 C_1$$

Substituting C_1,

$$C_2 = G_1 + P_1(G_0 + P_0C_0)$$

Therefore,

$$\boxed{C_2 = G_1 + P_1G_0 + P_1P_0C_0}$$

Similarly,

$$C_3 = G_2 + P_2C_2$$

Substituting C_2,

$$\boxed{C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0}$$

These equations are in **sum-of-products form**, so they can be implemented using one level of AND gates followed by an OR gate, or using a two-level NAND implementation.

The major advantage is that C_3does not need to wait for $C_2$and $C_1$to ripple through the previous stages. The carries can be generated in parallel. Hence, the carry lookahead adder performs addition faster.

The improvement in speed comes at the expense of **additional hardware and greater circuit complexity**.

Thus:

Ripple Carry Adder → Simple but Slow

Carry Lookahead Adder → Fast but More Complex

In a four-bit carry-lookahead adder, each sum output requires two XOR gates. The first XOR generates P_i, while the AND gate generates G_i. The carry lookahead generator produces the carry signals, which are then applied to the second XOR gate. The output carries are generated after only two levels of gates, giving much more uniform propagation delay for the sum outputs.

Practice Exercise 4.3

What is the main drawback of a ripple adder?

The main drawback is the time required to add long data words. The carry has to propagate from the **least significant bit to the most significant bit**, causing a large propagation delay.

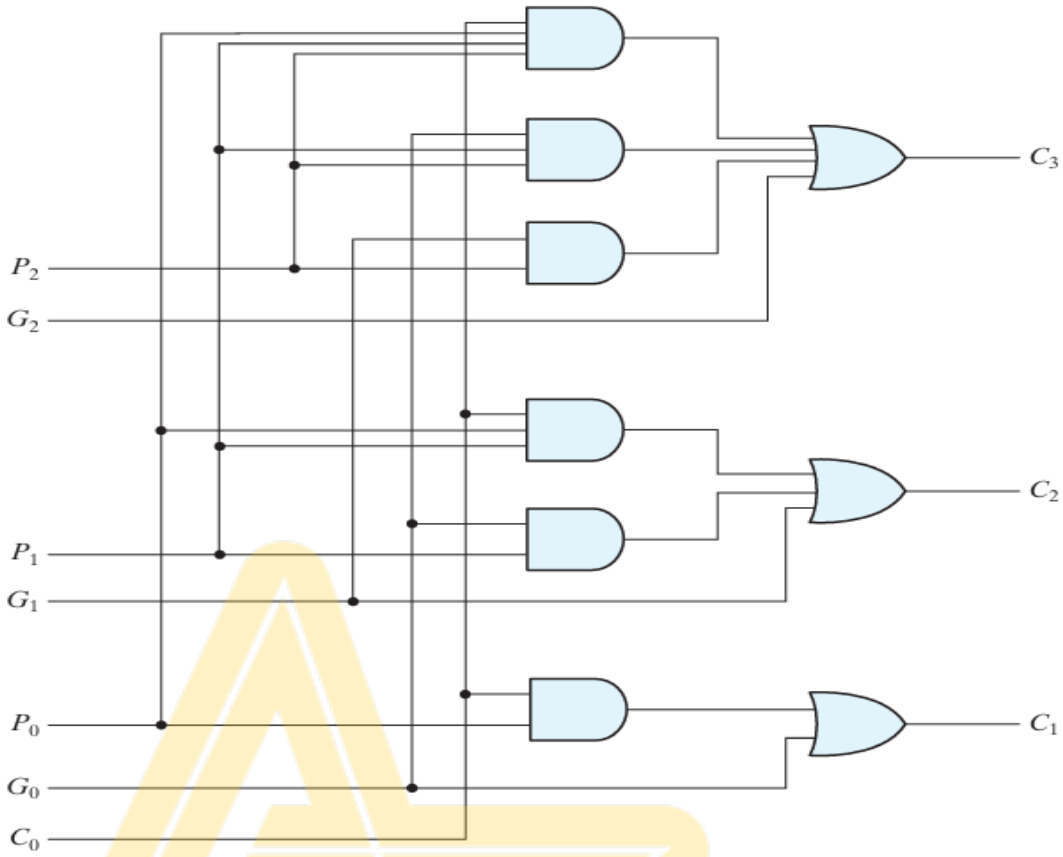


FIGURE 4.11
Logic diagram of carry lookahead generator

Main drawback = Carry propagation delay

Practice Exercise 4.4

What is the main drawback of a carry lookahead adder?

The main drawback is that its hardware is **more complex** than the hardware required for a ripple-carry adder.

Main drawback = More complex hardware

Practice Exercise 4.5

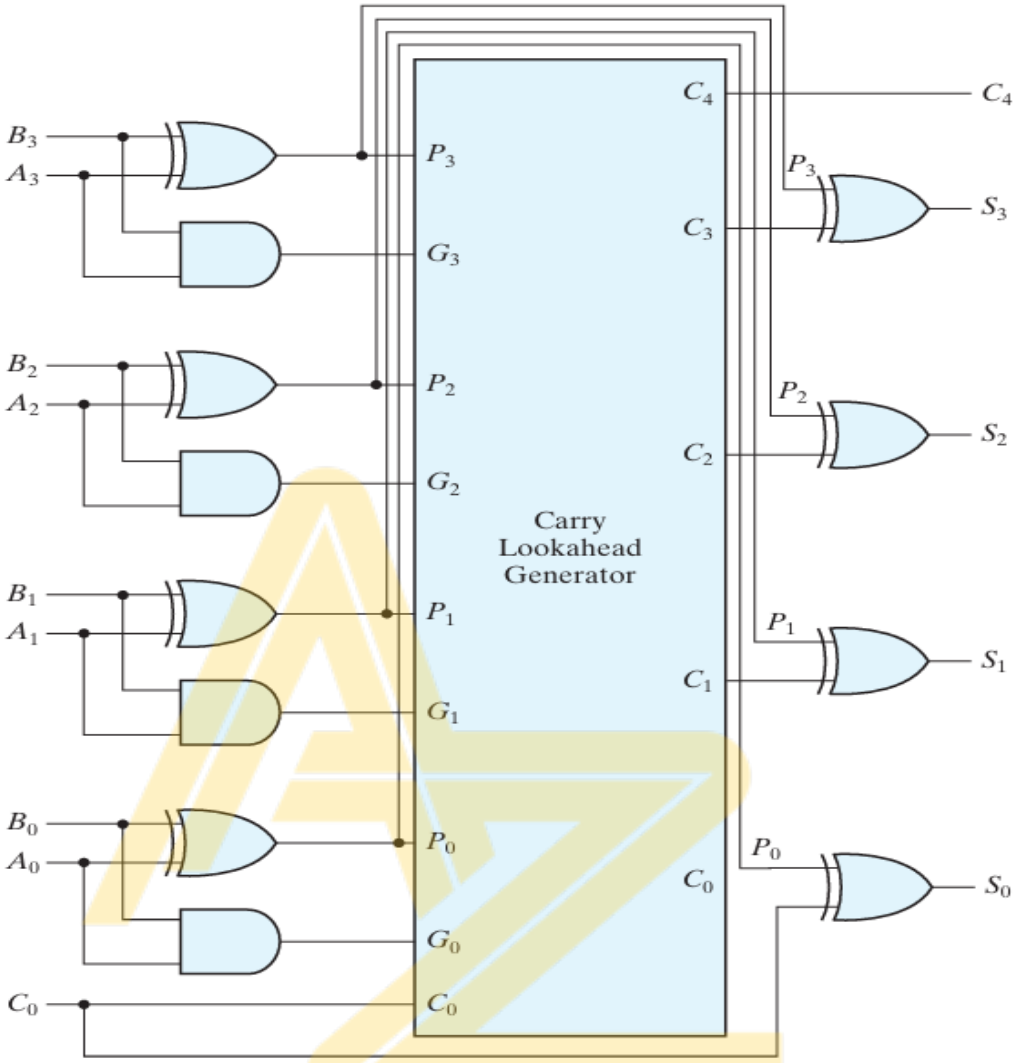


FIGURE 4.12
Four-bit adder with carry lookahead

Add:

$$A = 1100_0101$$
$$B = 1010_1010$$

The answer given in the textbook is:

$$\boxed{\text{Sum} = 0110_1111}$$

and

$$\boxed{\text{Carry} = 1}$$

2.5.8 BINARY SUBTRACTOR

Binary subtraction can conveniently be performed using **complements**. The subtraction

$$A - B$$

can be performed by taking the **2's complement of** B and adding it to A:

$$\boxed{A - B = A + (2's\ complement\ of\ B)}$$

The 2's complement is obtained by taking the 1's complement and then adding 1:

$$\boxed{2's\ complement = B' + 1}$$

The 1's complement can be implemented using inverters, while the additional 1 can be supplied through the input carry of the least significant full adder.

Therefore,

$$A - B = A + B' + 1$$

The subtractor consists of an adder with inverters placed between each B input and the corresponding full-adder input. When subtraction is performed,

$$\boxed{C_0 = 1}$$

This means the circuit performs

$$A + 1\text{'s complement of } B + 1$$

which is exactly

$$\boxed{A + 2\text{'s complement of } B}$$

and therefore performs

$$\boxed{A - B}$$

For unsigned numbers, this gives $A - B$ when

$$A \geq B$$

If

$$A < B$$

the result is the 2's complement representation of

$$B - A$$

For signed numbers, the result represents $A - B$, provided that no overflow occurs.

2.5.9 ADDER–SUBTRACTOR CIRCUIT

It is possible to combine addition and subtraction into **one common binary adder**. This is done by placing an XOR gate at each input B of the full adders. A control input called the **mode input** M determines whether the circuit performs addition or subtraction.

The two modes are:

$$\boxed{M = 0 \rightarrow \text{Addition}}$$
$$\boxed{M = 1 \rightarrow \text{Subtraction}}$$

When $M = 0$: Addition

Each XOR gate receives $M = 0$. Therefore,

$$B \oplus 0 = B$$

Also,

$$C_0 = 0$$

Hence, the full adders receive the original B bits and perform

$$\boxed{A + B}$$

Therefore,

$$\boxed{M = 0 \Rightarrow A + B}$$

When $M = 1$: Subtraction

Each XOR gate receives $M = 1$. Therefore,

$$B \oplus 1 = B'$$

Also,

$$C_0 = 1$$

Thus, all B inputs are complemented and 1 is added through the input carry. The circuit performs

$$A + B' + 1$$

Since

$$B' + 1 = 2's\ complement(B)$$

we obtain

$$\boxed{A + B' + 1 = A - B}$$

Therefore,

$$\boxed{M = 1 \Rightarrow A - B}$$

The XOR gate associated with output V is used to detect **overflow**.

An important point is that signed and unsigned binary numbers use the same basic hardware for addition and subtraction. The difference is in how the resulting binary value is interpreted.

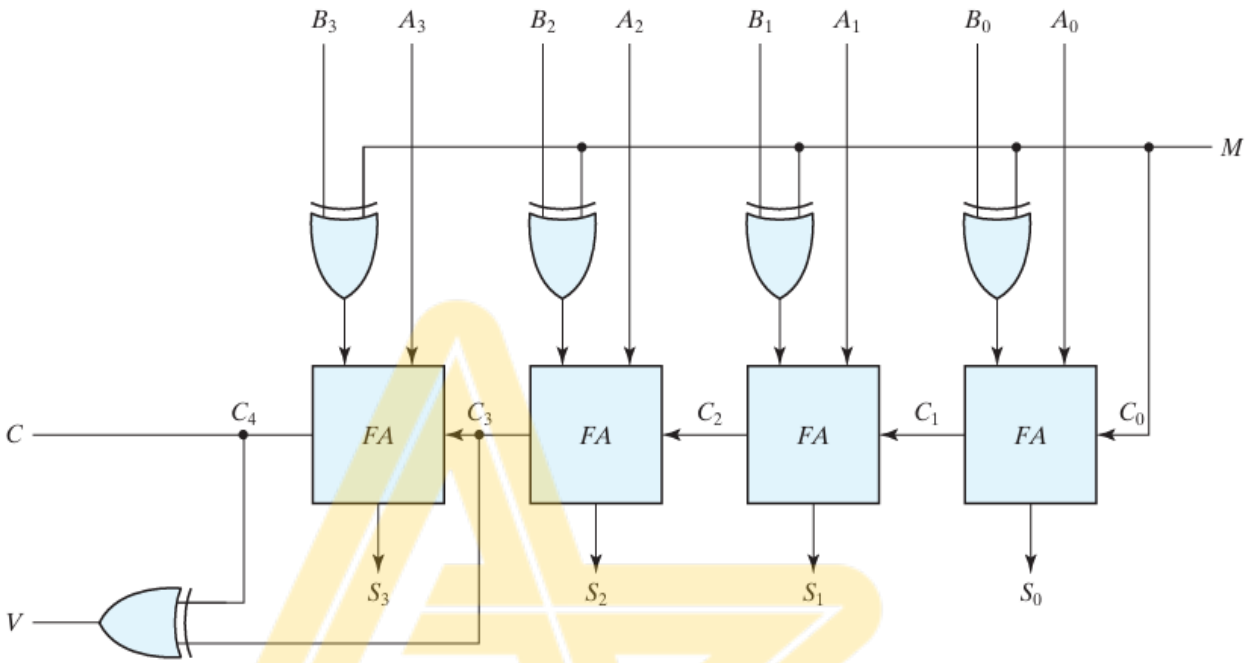


FIGURE 4.13
Four-bit adder–subtractor (with overflow detection)

Practice Exercise 4.6

Find

$$A - B$$

where

$$A = 1001_2$$

and

$$B = 0110_2$$

The textbook answer is

$$\boxed{A - B = 1_0011_2}$$

2.5.10 OVERFLOW

Overflow occurs when the result of an arithmetic operation requires more bits than are available to represent it.

If two numbers having $n$digits each are added and the resulting number requires $n + 1$digits, an overflow has occurred.

Thus,

$$\boxed{n\text{-bit result required} \rightarrow n + 1 \text{ bits}}$$

means that the result cannot fit into the available n-bit word.

Overflow is not normally a problem when doing calculations on paper because we can simply write another digit. In a digital computer, however, the number of bits available in a register is fixed. Therefore, an $n + 1$-bit result cannot be stored in an n-bit register. Computers can detect this condition and set an appropriate status or flip-flop so that the overflow can be detected.

2.5.11 UNSIGNED OVERFLOW AND SIGNED OVERFLOW

Overflow detection depends on whether the binary numbers are interpreted as **unsigned** or **signed**.

For unsigned numbers, overflow is detected from the **carry out of the most significant bit**.

$$\boxed{\text{Unsigned overflow} \Rightarrow C = 1}$$

For signed numbers, the leftmost bit represents the sign, and negative numbers are represented using **2's complement**. In signed addition, the final carry out does **not** indicate overflow because the sign bit is treated as part of the number.

2.5.12 WHEN CAN SIGNED OVERFLOW OCCUR?

If one number is positive and the other number is negative, overflow cannot occur during addition. This is because adding numbers with opposite signs produces a result whose magnitude is smaller than the magnitude of the larger original number.

However, overflow can occur when:

$$\boxed{\text{positive} + \text{positive}}$$

or

$$\boxed{\text{negative} + \text{negative}}$$

This is an extremely important point for examinations.

Example: $+70 + (+80)$

Consider two signed binary numbers:

$$+70$$

and

$$+80$$

stored in two 8-bit registers.

An 8-bit signed 2's-complement register can represent values from

$$-128$$

to

$$+127$$

But

$$70 + 80 = 150$$

and

$$150 > 127$$

Therefore, $+150$cannot be represented using 8 bits.

Hence,

Overflow occurs

The textbook also considers

$$-70 + (-80) = -150$$

Since

$$-150 < -128$$

this result also cannot be represented in an 8-bit signed register, so overflow occurs again.

The important observation is that the eight-bit result gets the **wrong sign**. A result that should be positive may have a negative sign bit, while a result that should be negative may have a positive sign bit. If the carry out of the sign-bit position were included, the resulting nine-bit number would be correct, but the nine-bit result cannot fit into the eight-bit register. Therefore, overflow has occurred.

2.5.13 DETECTING SIGNED OVERFLOW

Signed overflow can be detected by observing two carry signals:

$$C_{\text{in}} = \text{carry into the sign-bit position}$$

and

$$C_{\text{out}} = \text{carry out of the sign-bit position}$$

If these two carries are **different**, overflow has occurred.

Therefore,

$$\boxed{V = C_{\text{in}} \oplus C_{\text{out}}}$$

where V is the overflow output.

If the two carries are equal,

$$V = 0$$

and there is no overflow.

If they are different,

$$V = 1$$

and overflow has occurred. An XOR gate can therefore be used to detect the overflow condition.

Final Concept: C and V Outputs

The binary adder–subtractor has two particularly important outputs:

$$\boxed{C = \text{Carry/Borrow indication}}$$

and

$$\boxed{V = \text{Overflow indication}}$$

If the numbers are treated as **unsigned**, the C output detects a carry after addition or a borrow after subtraction.

If the numbers are treated as **signed**, the V output detects overflow.

Thus,

$$\boxed{V = 0 \Rightarrow \text{No signed overflow}}$$

and

$$\boxed{V = 1 \Rightarrow \text{Signed overflow}}$$

When $V = 1$, the actual result requires $n + 1$bits, but only the rightmost $n$bits can be stored in the available space.

2.6 DECODERS

A **decoder** is one of the most important combinational logic circuits because it is used to convert coded binary information into a form in which each possible input combination selects a unique output. The textbook develops the decoder from the idea of binary codes, then explains the **three-to-eight-line decoder**, decoders with **enable inputs**, their use as **demultiplexers**, cascading decoders to construct larger decoders, and finally the use of decoders for implementing **Boolean functions**. I will keep all of these points and examples, but explain the logic behind them more clearly.

2.6.1 BASIC IDEA OF A DECODER

Digital systems represent discrete information using **binary codes**. A binary code containing $n$bits can represent a maximum of

$$\boxed{2^n}$$

different elements of coded information.

A **decoder** is a combinational circuit that takes binary information from $n$input lines and converts it into a maximum of 2^nunique output lines.

Therefore,

$$\boxed{n \text{ inputs} \rightarrow 2^n \text{ maximum outputs}}$$

If some combinations of the n-bit code are not used, the decoder does not necessarily need all 2^noutputs. Therefore, a decoder can have fewer than 2^noutputs when some input combinations are unused.

Decoders are generally described as n**-to-**m**-line decoders**, where

$$\boxed{m \leq 2^n}$$

The main purpose of a decoder is to generate the 2^n, or fewer, **minterms of** n**input variables**.

The most important property is that **each input combination activates a unique output**. In other words, for every possible input combination, one particular output is selected.

The term *decoder* is also used for other types of code converters, such as a **BCD-to-seven-segment decoder**.

2.6.2 THREE-TO-EIGHT-LINE DECODER

Consider a **three-to-eight-line decoder**.

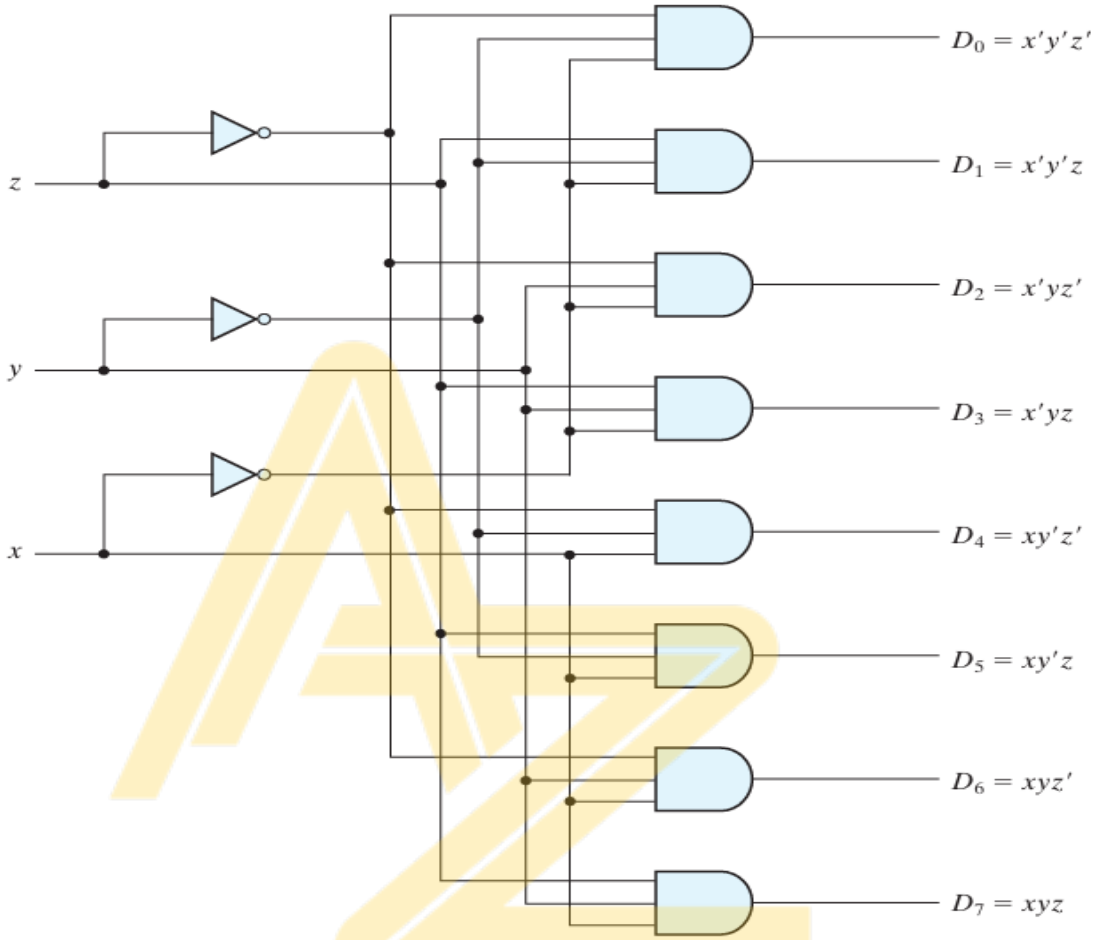


FIGURE 4.18
Three-to-eight-line decoder

Table 4.6
Truth Table of a Three-to-Eight-Line Decoder

Inputs			Outputs							
x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

It has three inputs:

$$x,\ y,\ z$$

Since there are three inputs, the maximum number of different combinations is

$$2^3 = 8$$

Therefore, the circuit has eight outputs:

$$D_0, D_1, D_2, D_3, D_4, D_5, D_6, D_7$$

Each output corresponds to exactly one of the eight possible combinations of x, y, z. In other words, the eight outputs represent the eight **minterms** of the three input variables.

The circuit uses three inverters to generate the complemented forms

$$x',\ y',\ z'$$

and eight AND gates. Each AND gate generates one minterm.

The eight minterms are:

$$\boxed{D_0 = x'y'z'}$$
$$\boxed{D_1 = x'y'z}$$
$$\boxed{D_2 = x'yz'}$$
$$\boxed{D_3 = x'yz}$$
$$\boxed{D_4 = xy'z'}$$
$$\boxed{D_5 = xy'z}$$
$$\boxed{D_6 = xyz'}$$
$$\boxed{D_7 = xyz}$$

The important idea is that **each AND gate receives a different combination of complemented and uncomplemented inputs**, so only one AND gate can produce 1 for any particular input combination.

For example, when

$$x = 0,\ y = 0,\ z = 0$$

we have

$$D_0 = x'y'z' = 1$$

while every other minterm is 0.

Similarly, when

$$x = 1,\ y = 1,\ z = 1$$

we get

$$D_7 = xyz = 1$$

while all other outputs are 0.

2.6.3 APPLICATION: BINARY-TO-OCTAL CONVERSION

One important application of a three-to-eight-line decoder is **binary-to-octal conversion**.

A three-bit binary number has eight possible combinations:

$$000,001,010,011,100,101,110,111$$

These correspond to the eight octal digits:

$$0,1,2,3,4,5,6,7$$

Therefore, the three input variables can represent a binary number, while the eight decoder outputs represent the corresponding octal digits.

However, the decoder is not restricted only to binary-to-octal conversion. It can be used to decode **any three-bit code** and provide eight separate outputs, one for each possible code element.

2.6.4 TRUTH TABLE OF A THREE-TO-EIGHT-LINE DECODER

The operation of the decoder becomes especially clear from its truth table.

x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0

x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

The key observation is:

Exactly one output is 1 at any time

and

All other outputs are 0

The output that becomes 1 represents the **minterm corresponding to the binary number currently present at the inputs**.

For example,

$$xyz = 010$$

corresponds to decimal 2, so

$$D_2 = 1$$

and all other outputs are 0.

Thus, the decoder effectively takes a binary input and **selects one corresponding output line**.

2.6.5 DECODER USING NAND GATES

Some decoders are constructed using **NAND gates** rather than AND gates.

A NAND gate performs the AND operation followed by inversion:

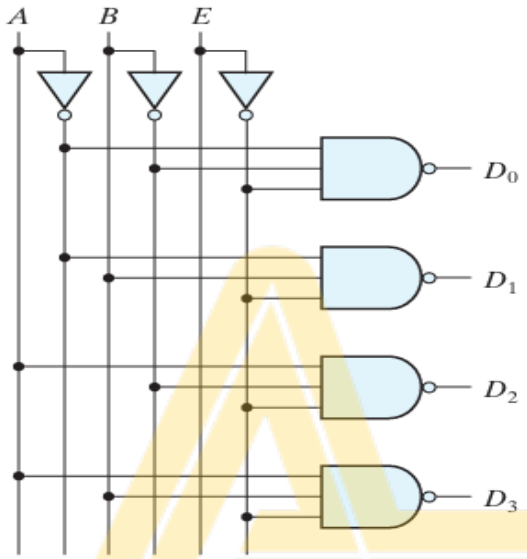
$$\text{NAND} = (AND)'$$

Because the output is inverted, it can be more economical to generate the **complemented forms of the decoder minterms**.

Decoders also commonly contain one or more **enable inputs**. The enable input controls whether the decoder is allowed to operate.

2.6.6 TWO-TO-FOUR-LINE DECODER WITH ENABLE

Consider a **two-to-four-line decoder with an enable input**.



E	A	B	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

FIGURE 4.19
Two-to-four-line decoder with enable input

There are two data inputs:

$$A,\ B$$

and one enable input:

$$E$$

There are four outputs:

$$D_0, D_1, D_2, D_3$$

Because there are two input variables,

$$2^2 = 4$$

possible combinations exist, so four outputs are required.

This particular decoder is constructed using NAND gates and has **active-low outputs** and an **active-low enable input**.

The phrase **active-low enable** means that the decoder is enabled when

$$\boxed{E = 0}$$

and disabled when

$$\boxed{E = 1}$$

When the decoder is enabled, only one output can become 0 at a time. All other outputs remain 1.

Therefore, this is the opposite convention from the earlier three-to-eight decoder:

Three-to-eight decoder

Selected output:

$$\boxed{1}$$

NAND-based active-low decoder

Selected output:

$$\boxed{0}$$

The selected output represents the minterm selected by $A$and B.

When $E = 0$

The decoder is enabled.

For example, if

$$AB = 10$$

then the corresponding output is selected:

$$\boxed{D_2 = 0}$$

while

$$D_0 = D_1 = D_3 = 1$$

Thus, exactly one output is active.

When $E = 1$

The decoder is disabled, regardless of the values of $A$and B.

Therefore,

$$\boxed{D_0 = D_1 = D_2 = D_3 = 1}$$

No minterm is selected.

2.6.7 ENABLE INPUTS

The enable input is an important feature of practical decoders.

A decoder may have:

Active-low enable

or

Active-high enable

Similarly, its outputs may be either:

Complemented

or

Uncomplemented

Some decoders also have **two or more enable inputs**. In that case, all enable inputs may have to satisfy a particular logic condition before the decoder becomes active.

Thus, the enable input provides a way of controlling whether the decoder should respond to its input variables.

2.6.8 DECODER AS A DEMULTIPLEXER

A decoder with an enable input can also function as a **demultiplexer**.

A demultiplexer is a circuit that receives information from a **single input line** and directs that information to one of 2^npossible output lines.

The selection of the output is controlled by $n$selection lines.

Therefore,

1 data input + n selection inputs → 2^n possible outputs

The two-to-four decoder discussed above can operate as a **one-to-four-line demultiplexer**.

In this configuration:

$$E = \text{data input}$$

and

$A, B =$ selection inputs

The single input E has a path to all four outputs, but the information is directed to only **one output line**, according to the binary combination of A and B.

For example, when

$$AB = 10$$

the output D_2 follows the input E, while the other outputs remain at their inactive value.

Thus,

$$\boxed{AB = 10 \Rightarrow D_2 \text{ carries the input information}}$$

This demonstrates why a decoder with an enable input is also referred to as a **decoder-demultiplexer**.

2.6.9 CONSTRUCTING A LARGER DECODER

Enable inputs also make it possible to connect smaller decoders together to create a larger decoder.

For example, two **3-to-8 decoders with enable inputs** can be connected together to form a

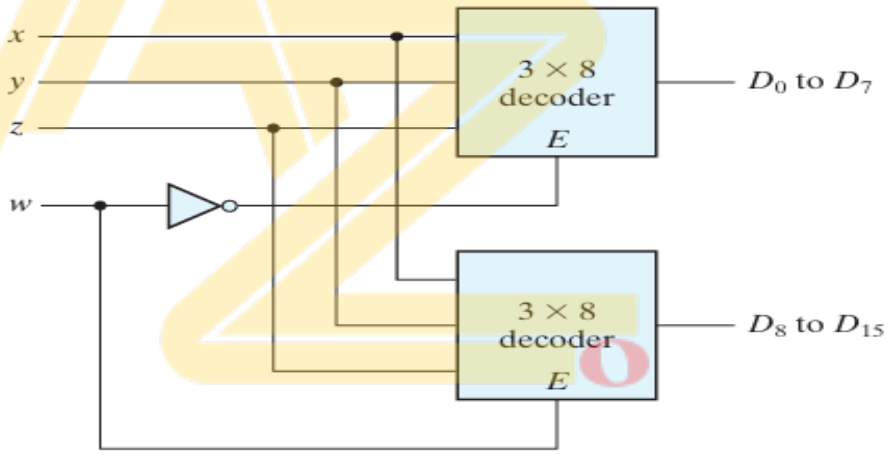


FIGURE 4.20
4 × 16 decoder constructed with two 3 × 8 decoders

4-to-16-line decoder

Suppose the four inputs are

$$w, x, y, z$$

The lower three inputs

$$x, y, z$$

are connected to both 3-to-8 decoders.

The fourth input $w$determines which decoder is enabled.

When $w = 0$

The **top decoder** is enabled, while the bottom decoder is disabled.

The top decoder generates the minterms

$$0000 \text{ to } 0111$$

Therefore, outputs

$$D_0 \text{ to } D_7$$

are generated.

The bottom decoder outputs remain inactive.

When $w = 1$

The enable conditions are reversed.

The **bottom decoder** becomes enabled, and it generates

$$1000 \text{ to } 1111$$

Therefore, outputs

$$D_8 \text{ to } D_{15}$$

are generated.

The top decoder outputs become inactive.

The important idea is that the enable inputs allow several standard decoder components to be interconnected to create a larger decoder.

In general,

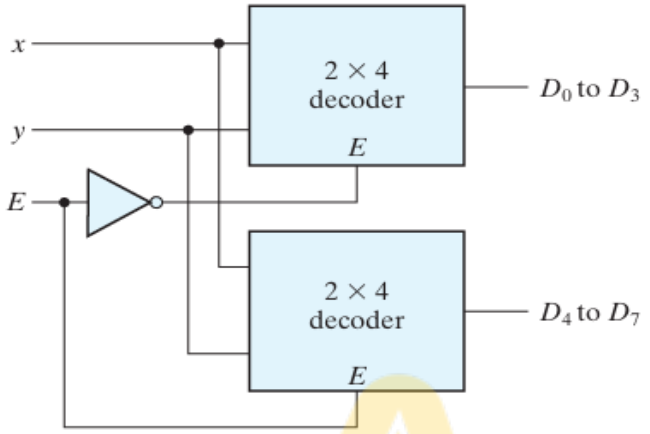
Enable inputs allow smaller standard components to be combined into larger systems.

This is useful not only for decoders but also for many other combinational logic components.

Practice Exercise 4.8

The textbook asks:

Draw a logic diagram constructing a 3×8decoder with active-low enable, using a pair of 2×4decoders. Also draw the truth table for the configuration.



E	x	y	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

FIGURE PE4.8

The basic idea is to use two 2-to-4 decoders. The two input variables are supplied to both decoders, while the third input controls which decoder is enabled.

The resulting eight outputs are

$$D_0, D_1, D_2, D_3, D_4, D_5, D_6, D_7$$

The first decoder produces D_0through D_3, while the second produces D_4through D_7. The active-low enable determines which group of four outputs becomes active.

This is exactly the same principle used earlier to construct the larger 4-to-16 decoder: **use enable inputs to select which smaller decoder operates**.

2.6.10 COMBINATIONAL LOGIC IMPLEMENTATION USING DECODERS

One of the most important applications of a decoder is the implementation of **Boolean functions**.

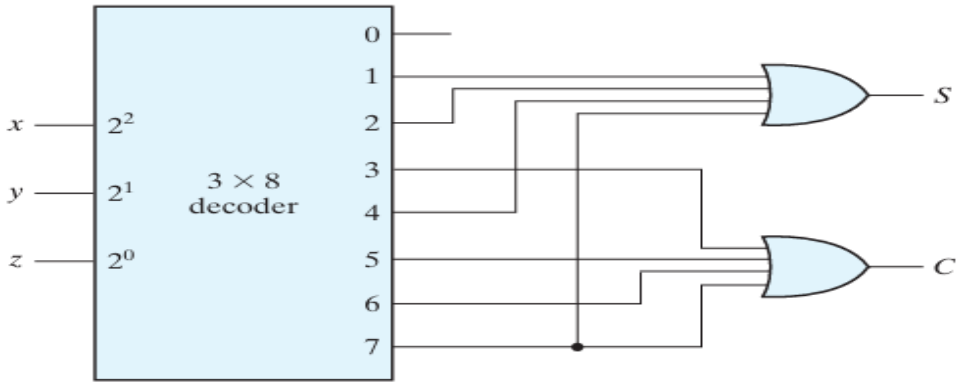


FIGURE 4.21
Implementation of a full adder with a decoder

A decoder generates the 2^n minterms of n input variables. Every possible input combination corresponds to exactly one decoder output.

Since **any Boolean function can be expressed as a sum of minterms**, a decoder can be used to generate the required minterms and an external OR gate can be used to combine them.

Therefore, a combinational circuit with

n inputs

and

m outputs

can be implemented using:

n-to-2^n decoder + m OR gates

The decoder generates all possible minterms, and each OR gate produces one output function by selecting the appropriate minterms.

2.6.11 PROCEDURE FOR IMPLEMENTING A BOOLEAN FUNCTION USING A DECODER

The Boolean function must first be expressed in **sum-of-minterms form**.

Then a decoder is selected that generates all the required minterms of the input variables.

Finally, the appropriate decoder outputs are connected to an OR gate according to the minterms required by each Boolean function.

Thus, the procedure is conceptually:

Boolean function

↓

Express as sum of minterms

↓

Select suitable decoder

↓

Connect required minterm outputs to OR gates

↓

Required Boolean function

2.6.12 EXAMPLE: IMPLEMENTING A FULL ADDER USING A DECODER

This is an **important textbook example**.

A full adder has three inputs:

$$x, y, z$$

Therefore, the total number of minterms is

$$2^3 = 8$$

The full-adder outputs are $S$and C.

From the full-adder truth table, the sum output is 1 for minterms

$$1,2,4,7$$

Therefore,

$$S(x, y, z) = \Sigma(1, 2, 4, 7)$$

The carry output is 1 for minterms

$$3,5,6,7$$

Therefore,

$$C(x, y, z) = \Sigma(3, 5, 6, 7)$$

Since there are three inputs and eight possible minterms, we require a

3-to-8 decoder

The decoder generates:

$$D_0, D_1, D_2, \dots, D_7$$

where each output corresponds to one minterm.

For the sum output, we need minterms

$$1,2,4,7$$

Therefore, these four decoder outputs are connected to an OR gate:

$$\boxed{S = D_1 + D_2 + D_4 + D_7}$$

For the carry output, we need minterms

$$3,5,6,7$$

Therefore,

$$\boxed{C = D_3 + D_5 + D_6 + D_7}$$

Hence, the complete implementation consists of one three-to-eight decoder and two OR gates.

This is a very important design principle:

$$\boxed{\text{Decoder generates minterms} + \text{OR gate selects required minterms} = \text{Boolean function}}$$

2.6.13 WHAT IF A FUNCTION HAS MANY MINTERMS?

Suppose a Boolean function has a very long list of minterms.

If the function contains $k$minterms, its complemented function can be expressed using the remaining

$$\boxed{2^n - k}$$

minterms.

Therefore, if

$$k > \frac{2^n}{2}$$

that is,

$$\boxed{k > 2^{n-1}}$$

then the complemented function contains fewer minterms than the original function.

In such a situation, it can be advantageous to implement the **complemented function** using fewer minterms. A NOR gate can then be used to sum the selected minterms and produce the normal function after the required inversion.

The basic idea is:

$$\boxed{\text{Many minterms} \rightarrow \text{Consider implementing the complement}}$$

because the complement may require fewer decoder outputs.

2.6.14 NAND DECODER AND EXTERNAL NAND GATES

When NAND gates are used to construct the decoder, the decoder outputs are complemented.

Therefore, the external gates used to combine the minterms must also be chosen appropriately.

Instead of using OR gates, **NAND gates** are used.

A two-level NAND implementation of a sum-of-minterms function is equivalent to a two-level AND-OR implementation. This is another application of DeMorgan's theorem and explains why NAND-based decoders are practical and economical.

2.7 ENCODERS

An **encoder** is a digital circuit that performs the **inverse operation of a decoder**. A decoder takes $n$input lines and produces up to 2^noutput lines, whereas an encoder takes 2^nor fewer input lines and produces $n$output lines. The output lines together generate the binary code corresponding to the active input. Thus, an encoder essentially converts the position of an active input line into its corresponding binary representation.

For example, an **octal-to-binary encoder** has **eight input lines**, one for each octal digit 0through 7, and **three output lines** because three binary bits are sufficient to represent eight different values:

$$2^3 = 8$$

The three outputs are x, y, z. It is assumed that **only one input is equal to 1 at any given time**.

Octal-to-Binary Encoder

Let the eight inputs be

$$D_0, D_1, D_2, D_3, D_4, D_5, D_6, D_7$$

and the three outputs be

$$x, y, z$$

When $D_0 = 1$, the output is 000, which represents binary 0. When $D_1 = 1$, the output is 001, representing binary 1. Similarly, D_2produces 010, D_3produces 011, and so on until D_7, which produces 111.

The important point is that the encoder does not need complicated logic for each output. We can directly inspect the truth table and determine which inputs must make each output equal to 1.

Output $z$is equal to 1 whenever the active octal digit is 1,3,5,or 7. Therefore,

$$z = D_1 + D_3 + D_5 + D_7$$

Output $y$is equal to 1 for octal digits 2,3,6,or 7. Therefore,

$$y = D_2 + D_3 + D_6 + D_7$$

Output $x$is equal to 1 for octal digits 4,5,6,or 7. Therefore,

$$x = D_4 + D_5 + D_6 + D_7$$

Hence, the encoder can be implemented using **three OR gates**, one for each output.

Limitation of the Basic Encoder

The octal-to-binary encoder described above has an important limitation: **only one input can be active at a time**.

Suppose two inputs become 1 simultaneously. The encoder does not know which input should be represented at the output, so the resulting output can become an undefined or ambiguous combination.

For example, suppose

$$D_3 = 1$$

and

$$D_6 = 1$$

at the same time.

From the Boolean equations,

$$z = D_1 + D_3 + D_5 + D_7$$

Since $D_3 = 1$,

$$z = 1$$

Similarly,

$$y = D_2 + D_3 + D_6 + D_7$$

Since $D_3 = 1$and $D_6 = 1$,

$$y = 1$$

And

$$x = D_4 + D_5 + D_6 + D_7$$

Since $D_6 = 1$,

$$x = 1$$

Therefore, the output becomes

$$xyz = 111$$

But 111represents binary 7. It does **not** represent either binary 3 or binary 6. Thus, the output is ambiguous.

To solve this problem, the encoder must establish a **priority** among the inputs. If we give higher priority to the input having the higher subscript number, then D_6has higher priority than D_3. Therefore, when both $D_3$and $D_6$are 1, $D_6$is encoded and the output becomes

$$110$$

which is binary 6.

This leads to the concept of a **priority encoder**.

Priority Encoder

A **priority encoder** is an encoder that includes a **priority function** so that it can properly handle the situation in which two or more inputs are active simultaneously.

Its operation is straightforward: when two or more inputs are equal to 1 at the same time, the input having the **highest priority** is selected, and its binary code appears at the output.

Table 4.7
Truth Table of an Octal-to-Binary Encoder

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

Table 4.8
Truth Table of a Priority Encoder

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

In the encoder considered here, the input with the **higher subscript number has higher priority**. Therefore,

$$D_3 > D_2 > D_1 > D_0$$

in terms of priority.

A four-input priority encoder has four inputs,

$$D_0, D_1, D_2, D_3$$

and two binary outputs,

$$x, y$$

because four possible inputs can be represented using two bits:

$$2^2 = 4$$

It also has a third output called the **valid-bit indicator**, represented by V.

The purpose of V is to tell us whether **at least one input is active**.

If all inputs are zero,

$$D_0 = D_1 = D_2 = D_3 = 0$$

then there is no valid input, so

$$V = 0$$

If at least one input is 1, then

$$V = 1$$

The valid bit is particularly important because an output of 00can otherwise have two possible meanings: it could mean $D_0 = 1$, or it could mean that **all inputs are 0**. The Voutput removes this ambiguity.

When

$$V = 0$$

the outputs $x$and $y$are not inspected because there is no valid input. Therefore, their values can be treated as **don't-care conditions**.

Understanding the Priority Operation

Because D_3has the highest priority, whenever

$$D_3 = 1$$

the output must be

$$xy = 11$$

regardless of the values of D_2, D_1,and D_0.

If

$$D_3 = 0$$

but

$$D_2 = 1$$

then D_2becomes the highest-priority active input, so the output is

$$xy = 10$$

regardless of $D_1$and D_0.

If both $D_3$and $D_2$are zero, then D_1can be encoded if it is 1:

$$xy = 01$$

Finally, if

$$D_3 = D_2 = D_1 = 0$$

and

$$D_0 = 1$$

then

$$xy = 00$$

The valid output is still

$$V = 1$$

because $D_0$is a valid active input.

If all four inputs are zero, then

$$V = 0$$

and the values of $x$and $y$are don't-care conditions.

Don't-Care Conditions in the Truth Table

The truth table of the priority encoder uses Xin two different ways, and it is important not to confuse them.

An Xappearing in an **output column** means that the output is a **don't-care condition**. In other words, its value does not matter for the operation of the circuit.

An Xappearing in an **input column** is different. It means that the input may be either 0 or 1. This is used to write the truth table in a condensed form rather than listing all 16 possible combinations of four inputs.

For example,

$$X100$$

represents two possible input combinations:

$$0100$$

and

$$1100$$

Thus, one row containing an Xcan represent more than one minterm.

For four input variables, there are

$$2^4 = 16$$

possible input combinations. The Xnotation allows these 16 combinations to be represented using fewer rows.

Boolean Functions of the Priority Encoder

The priority encoder can be implemented using the simplified Boolean functions obtained from the K-maps.

The output $x$is 1 whenever either D_2or $D_3$is the highest-priority active input. Therefore,

$$\boxed{x = D_2 + D_3}$$

The output $y$is 1 when $D_3$is active, or when $D_1$is active while the higher-priority input $D_2$is not active. The simplified expression is

$$y = D_3 + D_1 D_2'$$

The valid-bit output is simply 1 whenever at least one of the four inputs is 1. Therefore,

$$V = D_0 + D_1 + D_2 + D_3$$

These three equations completely describe the four-input priority encoder:

$$x = D_2 + D_3$$
$$y = D_3 + D_1 D_2'$$
$$V = D_0 + D_1 + D_2 + D_3$$

Thus, the circuit can be implemented using OR gates together with the required AND and NOT operations.

K-Map Understanding

The K-maps for $x$and $y$are obtained from the priority encoder truth table. Although the truth table shown in the textbook contains only a few rows, the X's in the input columns represent multiple actual input combinations.

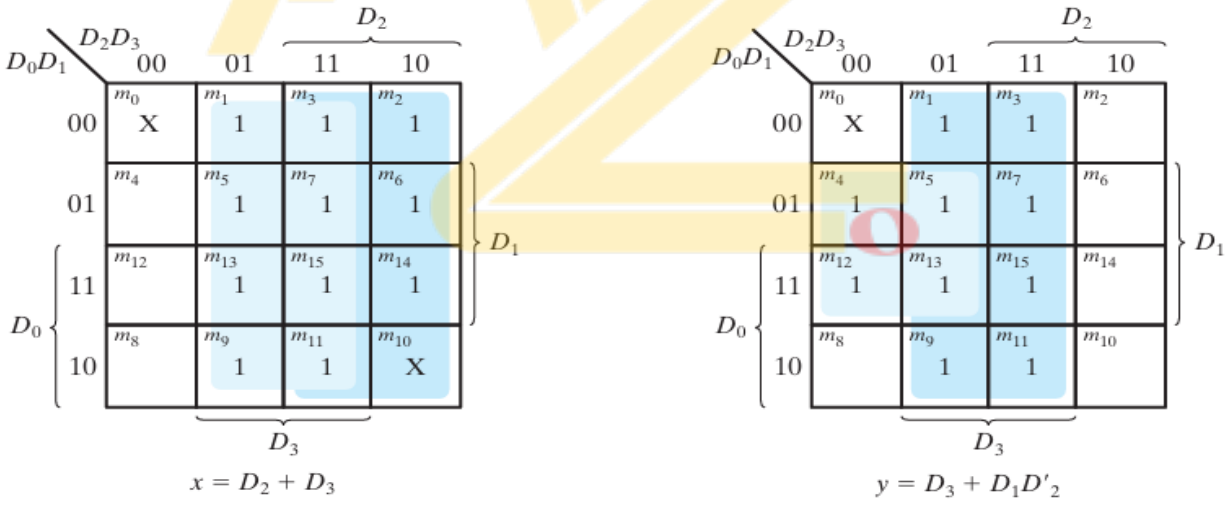


FIGURE 4.22
Maps for a priority encoder

For example, the row

$$XX10$$

represents the four minterms

$$0010,\ 0110,\ 1010,\ 1110$$

because each Xcan independently be either 0 or 1.

This is why the K-map can still account for all

$$2^4 = 16$$

possible combinations even though the condensed truth table contains only five rows.

The K-map simplification produces

$$x = D_2 + D_3$$

and

$$y = D_3 + D_1 D_2'$$

The valid-bit function does not require a K-map because its operation is directly an OR function:

$$V = D_0 + D_1 + D_2 + D_3$$

Four-Input Priority Encoder — Final Understanding

The complete operation can be remembered through the priority order

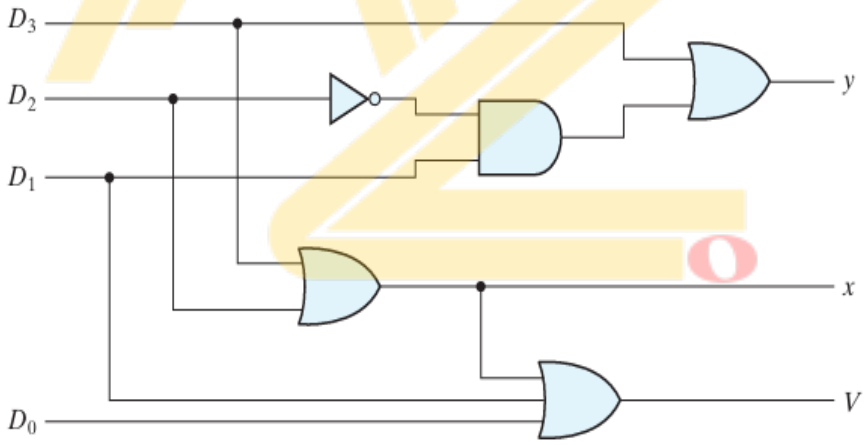


FIGURE 4.23
Four-input priority encoder

$$\boxed{D_3 > D_2 > D_1 > D_0}$$

If $D_3 = 1$, it wins regardless of all other inputs:

$$xy = 11$$

If $D_3 = 0$and $D_2 = 1$, then:

$$xy = 10$$

If $D_3 = D_2 = 0$and $D_1 = 1$, then:

$$xy = 01$$

If only $D_0$is active, then:

$$xy = 00$$

But to distinguish $D_0 = 1$from the condition in which **no input is active**, we use the valid bit:

$$V = D_0 + D_1 + D_2 + D_3$$

Therefore,

$$V = 1$$

means **at least one input is active**, while

$$V = 0$$

means **all inputs are inactive**.

The final Boolean equations given for the four-input priority encoder are:

$$\boxed{x = D_2 + D_3}$$
$$\boxed{y = D_3 + D_1 D_2'}$$
$$\boxed{V = D_0 + D_1 + D_2 + D_3}$$

4.8 MULTIPLEXERS

A **multiplexer**, commonly called a **MUX**, is a combinational circuit that selects binary information from **one of many input lines** and sends the selected information to **one output line**. The selection of which input should be connected to the output is controlled by a set of **selection lines**. In general, if a multiplexer has 2^ninput lines, it requires $n$selection lines. The different combinations of the selection lines determine which one of the input lines is selected.

The easiest way to understand a multiplexer is to think of it as an **electronic switch**. There may be many possible sources of data, but only one of them is allowed to reach the output at a particular time.

For example, if there are two input lines and one output, a selection line decides which of the two inputs will appear at the output.

2-to-1-Line Multiplexer

A **two-to-one-line multiplexer** has:

2 data inputs
1 output
1 selection line

Let the two data inputs be

$$I_0, I_1$$

and let the selection input be

$$S$$

with output

$$Y$$

When

$$S = 0$$

the upper AND gate is enabled, so I_0 gets a path to the output.

When

$$S = 1$$

the lower AND gate is enabled, so I_1 gets a path to the output.

Therefore,

$$S = 0 \Rightarrow Y = I_0$$

and

$$S = 1 \Rightarrow Y = I_1$$

So the multiplexer simply chooses between I_0 and I_1.

Its Boolean expression is

$$\boxed{Y = I_0 S' + I_1 S}$$

This equation is extremely important because it directly describes the operation of a 2-to-1 MUX.

When $S = 0$,

$$Y = I_0(1) + I_1(0)$$

so

$$Y = I_0$$

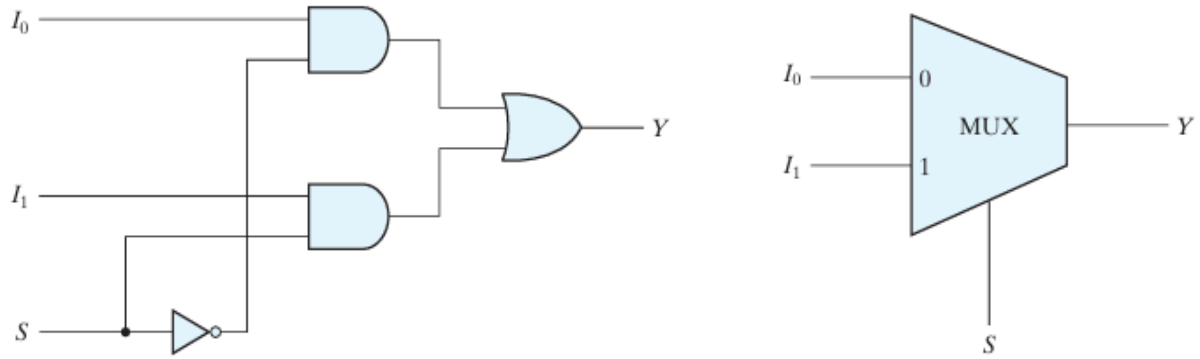
When $S = 1$,

$$Y = I_0(0) + I_1(1)$$

so

$$Y = I_1$$

Thus, the selection line determines which input reaches Y. The textbook also shows the MUX using a wedge-shaped block symbol, which visually represents multiple data sources being directed toward a single destination. The abbreviation **MUX** is commonly used in block diagrams.

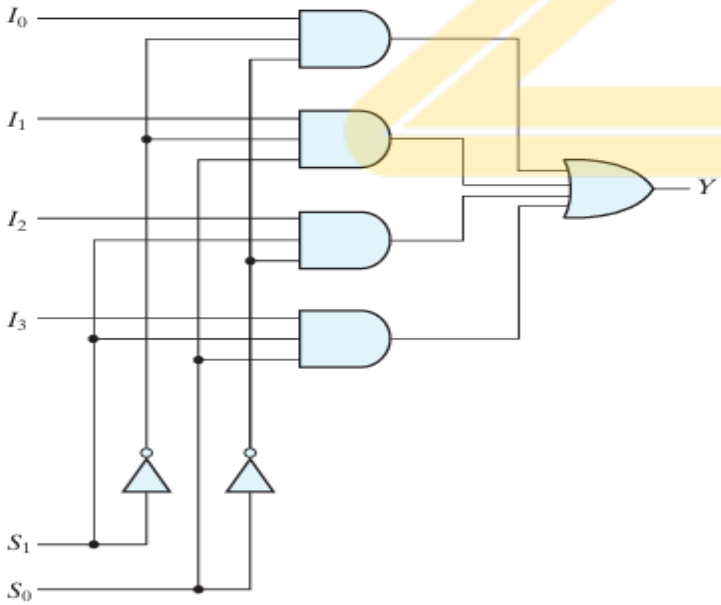


(a) Logic diagram (b) Block diagram

FIGURE 4.24
Two-to-one-line multiplexer

4-to-1-Line Multiplexer

A **four-to-one-line multiplexer** has four data inputs:



(a) Logic diagram

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(b) Function table

FIGURE 4.25
Four-to-one-line multiplexer

$$I_0, I_1, I_2, I_3$$

and two selection lines:

$$S_1, S_0$$

There is one output:

$$Y$$

Since there are four possible input lines,

$$2^2 = 4$$

two selection lines are sufficient to select any one of the four inputs.

The selection combinations are:

$$S_1S_0 = 00 \Rightarrow I_0$$
$$S_1S_0 = 01 \Rightarrow I_1$$
$$S_1S_0 = 10 \Rightarrow I_2$$
$$S_1S_0 = 11 \Rightarrow I_3$$

Internally, each data input is connected to an AND gate. The selection lines and their complements determine which AND gate becomes enabled. The outputs of all the AND gates are then connected to a single OR gate, producing the final output Y.

For a 4-to-1 MUX, the Boolean function is

$$\boxed{Y = I_0S_1'S_0' + I_1S_1'S_0 + I_2S_1S_0' + I_3S_1S_0}$$

Each term corresponds to one possible selection combination.

For example, consider

$$S_1S_0 = 10$$

Only the AND gate associated with $I_2$is enabled. The other three AND gates have at least one input equal to 0, so their outputs are 0. Consequently, the OR gate receives I_2as the only possible active input, and therefore

$$Y = I_2$$

This is why a multiplexer is also called a **data selector**: it selects one of many input lines and steers its binary information to the output.

MUX and Decoder Relationship

The AND gates and inverters inside a multiplexer are very similar to the gates used in a decoder. In fact, the selection inputs are effectively **decoded** to determine which data input gets connected to the output.

In general, a

$$2^n\text{-to-1-line MUX}$$

can be constructed using an

$$n\text{-to-}2^n \text{ decoder}$$

by adding the 2^ndata inputs to the decoder's AND gates. The outputs of these AND gates are then connected to one OR gate.

Therefore, the basic relationship is:

$$\boxed{n \text{ selection lines} \Rightarrow 2^n \text{ data inputs}}$$

The size of a multiplexer is normally specified by the number of its data inputs and its single output.

For example:

$$2\text{: 1 MUX} \Rightarrow 2 \text{ inputs, 1 output}$$
$$4\text{: 1 MUX} \Rightarrow 4 \text{ inputs, 1 output}$$
$$8\text{: 1 MUX} \Rightarrow 8 \text{ inputs, 1 output}$$

and so on.

Like decoders, multiplexers may also contain an **enable input**. When the enable input is in its inactive state, the outputs are disabled. When it is active, the multiplexer operates normally.

Multiple-Bit Selection Using Multiplexers

Multiplexers do not have to work with only one-bit data. Several multiplexers can be combined so that they use the **same selection line** to select an entire group of bits.

The textbook gives the example of a **quadruple 2-to-1-line multiplexer**. This circuit contains four separate 2-to-1 multiplexers. Each one selects between two input bits.

There are two 4-bit sets of inputs:

$$A_0, A_1, A_2, A_3$$

and

$$B_0, B_1, B_2, B_3$$

The outputs are

$$Y_0, Y_1, Y_2, Y_3$$

All four multiplexers share the same selection input S.

When the circuit is enabled and

$$S = 0$$

the four Ainputs are selected:

$$Y_0 = A_0,\ Y_1 = A_1,\ Y_2 = A_2,\ Y_3 = A_3$$

When

$$S = 1$$

the four Binputs are selected:

$$Y_0 = B_0,\ Y_1 = B_1,\ Y_2 = B_2,\ Y_3 = B_3$$

The textbook's enable input is active when

$$E = 0$$

and when

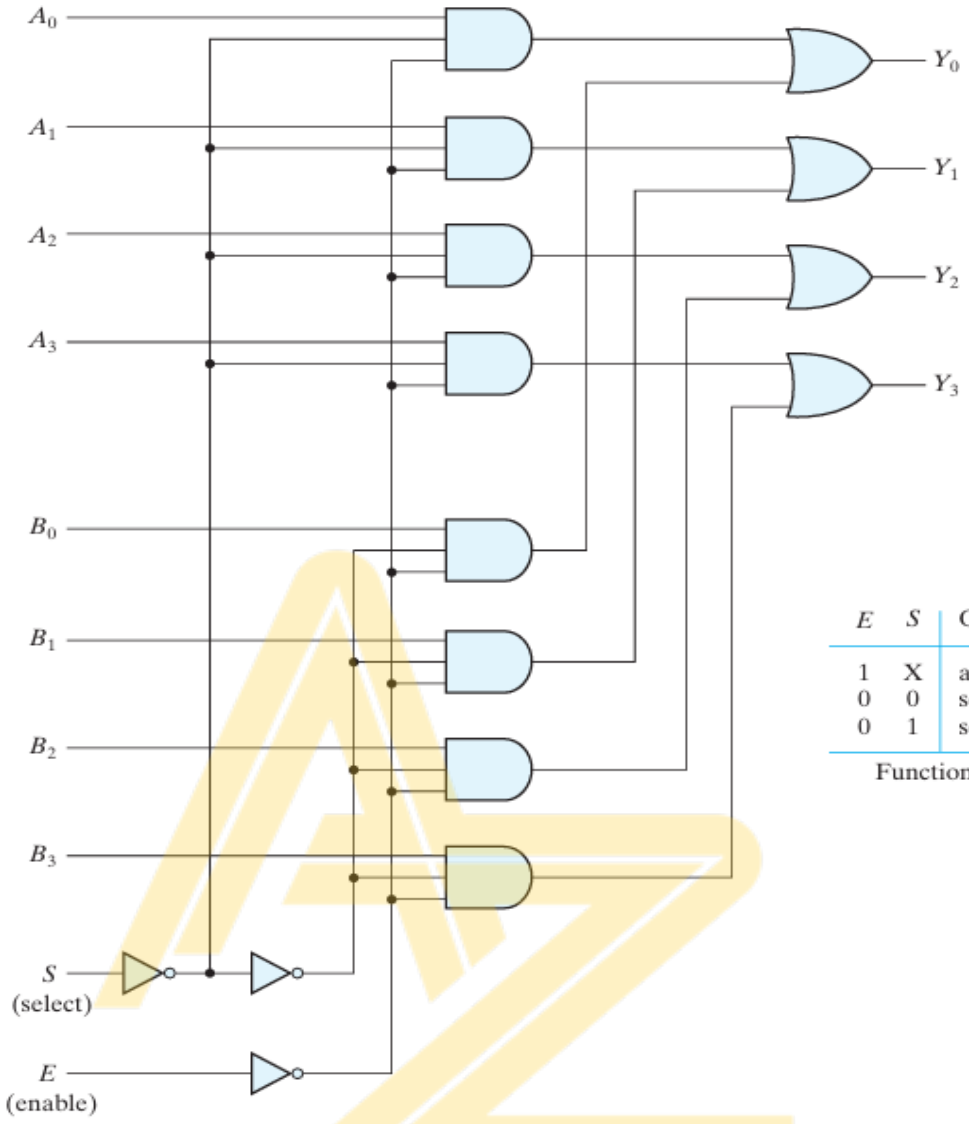
$$E = 1$$

the outputs become all 0's regardless of the value of S.

So instead of thinking of this as four independent multiplexers, it is often easier to view it as a circuit that chooses **one of two 4-bit data sets**.

Boolean Function Implementation Using Multiplexers

A very important application of multiplexers is the implementation of **Boolean functions**.



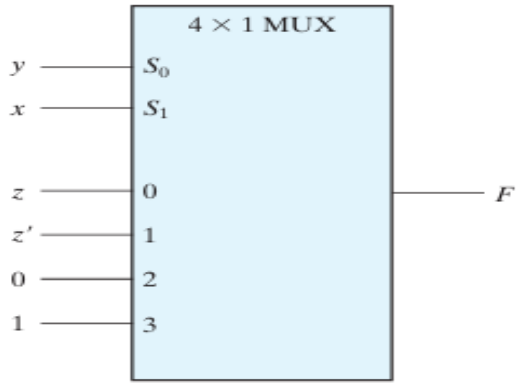
E	S	Output Y
1	X	all 0's
0	0	select A
0	1	select B

Function table

FIGURE 4.26
Quadruple two-to-one-line multiplexer

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

(a) Truth table



(b) Multiplexer implementation

FIGURE 4.27
Implementing a Boolean function with a multiplexer

Earlier, a decoder was used to implement a Boolean function by generating its minterms and then using external OR gates. A multiplexer provides a similar capability, but the OR operation is already included inside the MUX. Therefore, a multiplexer can be used to implement Boolean functions efficiently.

For an n-variable Boolean function, a direct method uses a multiplexer having

$$n \text{ selection inputs}$$

and

$$2^n \text{ data inputs}$$

Each data input corresponds to one minterm.

However, the textbook presents a **more efficient method** that uses only

$$n - 1$$

selection inputs and

$$2^{n-1}$$

data inputs.

The first $n - 1$variables are connected to the selection inputs. The remaining variable is used to determine the values of the data inputs.

If the remaining variable is z, each data input will be one of only four possibilities:

$$\boxed{0,\ 1,\ z,\ z'}$$

This is a very important MUX design technique.

Example: Implementing $F(x, y, z) = \Sigma(1, 2, 6, 7)$

Consider the Boolean function

$$\boxed{F(x, y, z) = \Sigma(1, 2, 6, 7)}$$

This is a three-variable Boolean function.

Since

$$n = 3$$

the efficient MUX method requires

$$n - 1 = 2$$

selection lines.

Therefore, we use a

$$\boxed{\text{4: 1 MUX}}$$

because

$$2^{n-1} = 2^2 = 4$$

The first two variables, $x$and y, are connected to the selection inputs:

$$S_1 = x$$
$$S_0 = y$$

The remaining variable is z, which will be connected to the data inputs in the appropriate form.

Now examine each combination of xy.

When $xy = 00$

The corresponding minterms are 0 and 1.

The function contains minterm 1 but not minterm 0. Therefore,

$$F = 0 \text{ when } z = 0$$

and

$$F = 1 \text{ when } z = 1$$

This is exactly the behavior of z. Therefore,

$$I_0 = z$$

When $xy = 01$

The corresponding minterms are 2 and 3.

Minterm 2 is included, while minterm 3 is not.

Therefore,

$$F = 1 \text{ when } z = 0$$

and

$$F = 0 \text{ when } z = 1$$

This is the behavior of

$$z'$$

Therefore,

$$I_1 = z'$$

When $xy = 10$

The corresponding minterms are 4 and 5.

Neither 4 nor 5 appears in

$$\Sigma(1, 2, 6, 7)$$

Therefore,

$$F = 0$$

for both values of z.

Hence,

$$I_2 = 0$$

When $xy = 11$

The corresponding minterms are 6 and 7.

Both are included in the function.

Therefore,

$$F = 1$$

for both values of z.

Hence,

$$I_3 = 1$$

Therefore, the required data inputs are

$$\boxed{I_0 = z}$$
$$\boxed{I_1 = z'}$$
$$\boxed{I_2 = 0}$$
$$\boxed{I_3 = 1}$$

with

$$S_1 = x,\ S_0 = y$$

This example demonstrates all four possible data-input forms:

$$\boxed{z,\ z',\ 0,\ 1}$$

which is exactly what the textbook points out.

General Procedure for Implementing a Boolean Function Using a MUX

For a Boolean function of $n$variables, use a multiplexer having

$$n - 1$$

selection inputs and

$$2^{n-1}$$

data inputs.

First, write the Boolean function in the form of a truth table. Then connect the first $n - 1$variables to the selection inputs of the multiplexer. For every possible combination of these selection variables, examine the function's value as a function of the final variable.

The resulting function of the final variable will always be one of:

$$\boxed{0,\ 1,\ z,\ z'}$$

These values are then connected to the corresponding MUX data inputs in the proper order.

This is one of the most useful procedures to remember for examinations.

Second Example: Four-Variable Function

The textbook next considers

A	B	C	D	F	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	

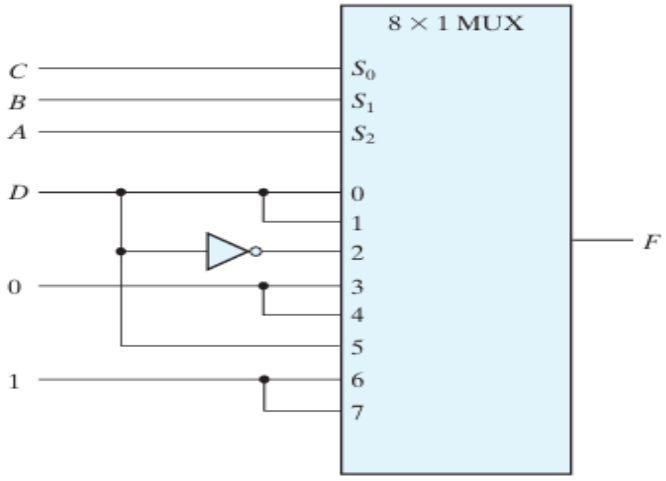


FIGURE 4.28
Implementing a four-input function with a multiplexer

$$F(A, B, C, D) = \Sigma(1, 3, 4, 11, 12, 13, 14, 15)$$

Since this function has four variables,

$$n = 4$$

the efficient method uses

$$n - 1 = 3$$

selection inputs.

Therefore, an

8: 1 MUX

is used.

The first three variables are connected to the selection lines in the order

$$A \rightarrow S_2,\ B \rightarrow S_1,\ C \rightarrow S_0$$

The remaining variable $D$is used for the data inputs.

The data-input number is determined directly from the binary combination ABC.

For example, when

$$ABC = 101$$

the corresponding data input is

$$I_5$$

The truth table shows that for this combination,

$$F = D$$

Therefore,

$$\boxed{I_5 = D}$$

The other data inputs are determined in exactly the same way. Depending on the corresponding values in the truth table, each data input can become

$$D,\ D',\ 0,\ \text{or}\ 1$$

The constants 0 and 1 correspond to fixed logic signals. In an integrated circuit, logic 0 can correspond to ground, while logic 1 corresponds to the power-supply signal, depending on the technology being used.

Practice Exercise 4.9

The textbook asks:

Answer:

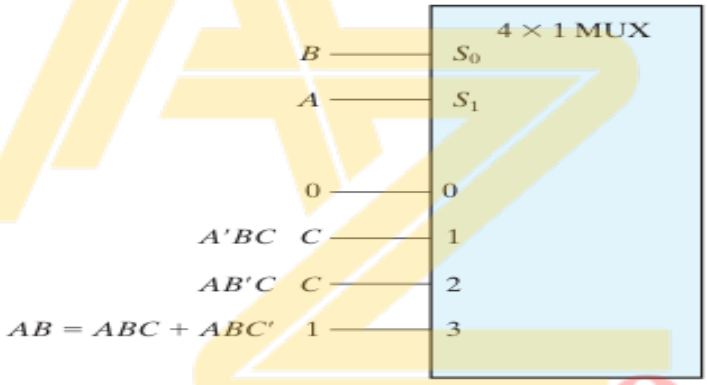


FIGURE PE4.9

$$\boxed{F(A,B,C) = \Sigma(3,5,6,7)}$$

Implement this Boolean function using a multiplexer.

Since there are three variables, use a

$$\boxed{4{:}\,1\ \text{MUX}}$$

with two selection variables. Taking $A$and Bas the selection inputs,

$$S_1 = A,\ S_0 = B$$

The remaining variable is C.

For $AB = 00$, the corresponding minterms are 0 and 1. Neither is present, so

$$I_0 = 0$$

For $AB = 01$, the corresponding minterms are 2 and 3. Only minterm 3 is present, so

$$I_1 = C$$

For $AB = 10$, the corresponding minterms are 4 and 5. Only minterm 5 is present, so

$$I_2 = C$$

For $AB = 11$, the corresponding minterms are 6 and 7. Both are present, so

$$I_3 = 1$$

Thus,

$$\boxed{I_0 = 0,\ I_1 = C,\ I_2 = C,\ I_3 = 1}$$

with

$$\boxed{S_1 = A,\ S_0 = B}$$

This is the MUX implementation represented in the textbook's Practice Exercise 4.9.

Three-State Gates

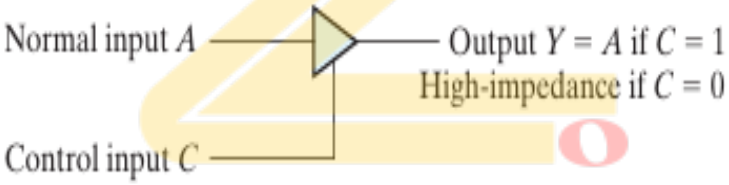


FIGURE 4.29
Graphic symbol for a three-state buffer

A multiplexer can also be constructed using **three-state gates**. A three-state digital circuit has three possible output states.

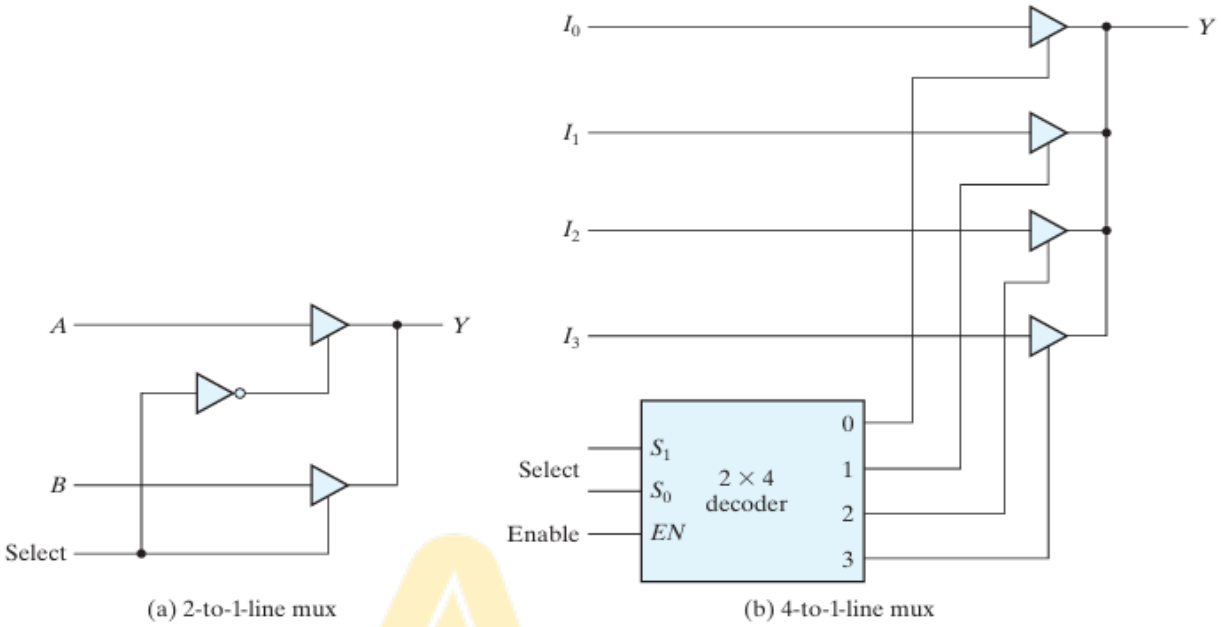


FIGURE 4.30
Multiplexers with three-state gates

The first two states are the familiar logic states:

0

and

1

The third state is called the **high-impedance state**, usually written as

$$Z$$

In the high-impedance state, the output behaves approximately like an **open circuit**. The output is effectively disconnected, so it has no logic significance and does not affect the circuit connected to it.

Three-state gates can implement conventional logic functions such as AND and NAND, but the most commonly used three-state gate is the **buffer**.

A three-state buffer has:

1. a normal input,
2. an output, and
3. a control input.

The control input determines whether the buffer is enabled.

When

$$C = 1$$

the output is enabled and the buffer behaves normally:

$$Y = A$$

When

$$C = 0$$

the output is disabled and enters the high-impedance state:

$$Y = Z$$

regardless of the value of A.

Why High-Impedance State Is Useful

The special advantage of a three-state gate is that many three-state outputs can be connected to a **common line**, often called a bus.

However, they must be controlled carefully. At any given time, only **one** three-state gate should be enabled. All other gates must remain in the high-impedance state.

This allows several circuits to share the same physical output line without interfering with one another.

This property is extremely important in digital systems because it allows multiple sources to share a common data bus.

Multiplexer Using Three-State Buffers

A 2-to-1 MUX can be constructed using two three-state buffers and an inverter.

The two buffer outputs are connected together to form a single output line. This type of connection is possible because the buffers can enter the high-impedance state.

When

$$S = 0$$

the upper buffer is enabled and the lower buffer is disabled. Therefore,

$$Y = A$$

When

$$S = 1$$

the lower buffer is enabled and the upper buffer is disabled. Therefore,

$$Y = B$$

Thus, the circuit performs exactly the same function as a conventional 2-to-1 MUX.

A 4-to-1 MUX can similarly be constructed using four three-state buffers. Their outputs are connected to a common output line. The control inputs ensure that only the selected buffer is enabled while the others remain in the high-impedance state.

A decoder can be used to ensure that no more than one buffer is active at a time. When the decoder is disabled, all four buffer control signals are inactive and the bus remains in the high-impedance state. When the decoder is enabled, exactly one buffer becomes active according to the binary value applied to the selection inputs.

SYNCHRONOUS SEQUENTIAL LOGIC

2.9 INTRODUCTION

Almost every modern electronic device such as mobile phones, computers, digital cameras, navigation systems, and media players needs to **store, retrieve, and process binary information**.

For example, a computer must remember information while executing a program. A mobile phone must remember data, settings, instructions, and intermediate results. Therefore, digital systems need electronic components that can **store information**.

The major idea introduced in this chapter is therefore **memory**.

Until now, we mainly studied **combinational circuits**. In a combinational circuit, the output depends only on the **present input values**. The circuit does not remember what the inputs were previously.

For example, if a combinational circuit has inputs $A$and B, its output can be represented as

$$F = f(A, B)$$

The previous values of $A$and Bdo not matter.

A **sequential circuit**, on the other hand, has memory. Its output depends not only on the current inputs but also on information that was stored from the past.

Therefore, the fundamental distinction is:

Combinational circuit = no memory
Sequential circuit = memory

A sequential circuit can **store, retain, and later retrieve information**. Understanding this distinction is essential before studying the rest of the chapter.

2.10 Sequential Circuits

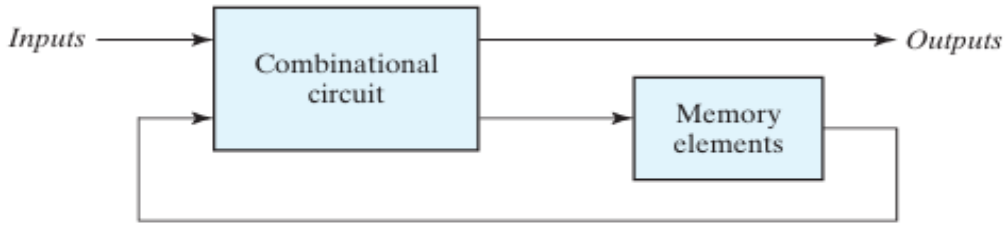


FIGURE 5.1
Block diagram of sequential circuit

A sequential circuit consists of two major parts:

$$\boxed{\text{Combinational circuit} + \text{Memory elements}}$$

The memory elements are connected to the combinational circuit in such a way that a **feedback path** is formed.

The basic idea can be represented as:

$$\text{Inputs} \rightarrow \text{Combinational Logic} \rightarrow \text{Outputs}$$

while the memory elements store the current state and feed information back into the combinational logic.

The memory elements are capable of storing **binary information**. The information stored in the memory elements at a particular instant is called the **state** of the sequential circuit.

Therefore,

$$\boxed{\text{State} = \text{binary information currently stored in the memory elements}}$$

The external inputs and the present state together determine the outputs.

Thus,

$$\boxed{\text{Output} = f(\text{present input}, \text{present state})}$$

The external inputs and present state also determine what the **next state** will be:

$$\boxed{\text{Next state} = f(\text{present input}, \text{present state})}$$

This is the key difference from a combinational circuit.

For a combinational circuit:

$$\boxed{\text{Output} = f(\text{present inputs})}$$

For a sequential circuit:

$$\boxed{\text{Output} = f(\text{present inputs, present state})}$$

and the state itself can change with time.

Therefore, a sequential circuit is specified by a **time sequence of inputs, outputs, and internal states**.

Combinational vs Sequential Circuit

Think of the difference this way.

Suppose you have a circuit whose output is determined only by what you give it **right now**. That is a combinational circuit.

If the circuit can remember something that happened **earlier** and use that information along with the current input, it is a sequential circuit.

So:

$$\boxed{\text{Combinational: Present input} \rightarrow \text{Output}}$$

whereas

$$\boxed{\text{Sequential: Present input + Past information} \rightarrow \text{Output}}$$

The "past information" is represented by the **present state stored in memory elements**.

Two Types of Sequential Circuits

Sequential circuits are classified according to the **timing of their signals**.

There are two main types:

$$\boxed{\text{Synchronous sequential circuits}}$$

and

$$\boxed{\text{Asynchronous sequential circuits}}$$

Asynchronous Sequential Circuits

In an **asynchronous sequential circuit**, the behavior depends on the input signals at any instant of time and also on the **order in which the inputs change**.

These circuits commonly use **time-delay devices** as storage elements.

The storage capability comes from the time required for a signal to propagate through a device. In practical gate-based asynchronous systems, the propagation delay of logic gates themselves can provide the required delay, so separate delay units may not always be necessary.

An asynchronous sequential circuit can therefore be viewed approximately as:

Combinational circuit + Feedback

The feedback between logic gates can cause the circuit to become **unstable** at certain times.

This instability creates significant design difficulties and limits the practical use of asynchronous sequential circuits. Because of these difficulties, the textbook does not cover asynchronous sequential circuits further.

Synchronous Sequential Circuits

A **synchronous sequential circuit** is different because its behavior can be described at **discrete instants of time**.

The storage elements are affected only at specific times determined by a timing signal called the **clock**.

Synchronization is provided by a device called a **clock generator**, which produces a periodic sequence of clock pulses.

The clock signal is commonly represented by:

clock

or

clk

The clock pulses are distributed throughout the system so that the storage elements are updated only when the appropriate clock pulse occurs.

Therefore, the clock essentially determines:

When should the stored information change?

The other signals, such as external inputs, determine:

What should the new information be?

Simple Example

Suppose we have a circuit that adds two binary numbers and stores the result.

The combinational logic first calculates

$$A + B$$

But the result is not stored continuously. When the appropriate clock pulse arrives, the calculated result is stored in the memory elements.

So the clock determines **when the storage happens**.

This is why circuits that use clock pulses to control their storage elements are called **clocked sequential circuits**.

They are the most commonly encountered type of sequential circuit in practice.

Why Synchronous Circuits Are Easier to Design

Synchronous sequential circuits are particularly useful because their activity is divided into **discrete time steps**.

Instead of worrying about changes happening at every possible instant, the designer can consider what happens from one clock pulse to the next.

The basic sequence is:

Present state → Combinational processing → Next state

The next state is stored when the appropriate clock pulse occurs.

Because synchronous circuits generally have fewer instability problems and their timing can be broken into discrete steps, they are much easier to design than asynchronous sequential circuits.

Flip-Flops — The Memory Elements

The storage elements used in clocked sequential circuits are called **flip-flops**.

A **flip-flop is a binary storage device capable of storing one bit of information**.

Since one bit can have only two possible values, a flip-flop can store either

0

or

1

Thus, in its stable state, the output of a flip-flop is either

$$Q = 0$$

or

$$\boxed{Q = 1}$$

A sequential circuit can use many flip-flops to store many bits.

For example, to store a 64-bit word, we can use 64 bits of storage:

$$\boxed{\text{64 bits of information} \rightarrow \text{64 bits of storage}}$$

The next sections of this chapter will study different types of flip-flops and latches in detail.

Synchronous Clocked Sequential Circuit

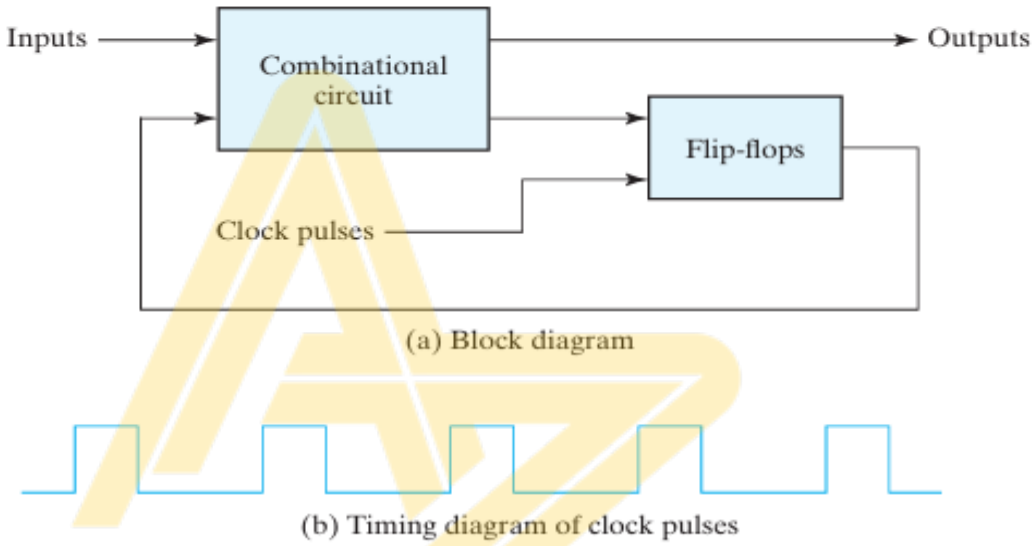


FIGURE 5.2
Synchronous clocked sequential circuit

A synchronous clocked sequential circuit contains:

- Inputs
- Combinational circuit
- Flip-flops
- Outputs

and a

- Clock

The outputs are generated by a combinational logic function of the external inputs and/or the values currently stored in the flip-flops.

Therefore,

$$\boxed{\text{Output} = f(\text{Inputs, Flip-flop values})}$$

The value that will be stored in the flip-flops at the next clock pulse is also determined by the external inputs and/or the present flip-flop values.

So,

$$\boxed{\text{Next state} = f(\text{Inputs, Present state})}$$

When a clock pulse occurs, the flip-flops are updated and the new values are stored.

Why Combinational Logic Must Be Fast Enough

This is a very important timing concept.

Before the next clock pulse arrives, the combinational logic must have enough time to calculate the correct value that needs to be stored in the flip-flops.

Suppose the clock pulses occur periodically:

$$\boxed{\text{Clock pulse}_1\ \text{Clock pulse}_2\ \text{Clock pulse}_3\ \cdots}$$

After one clock pulse changes the flip-flop state, the combinational circuit needs some time to respond to that new state.

It must calculate the next value before the following clock pulse arrives.

Therefore,

$$\boxed{\text{Combinational logic must settle before the next clock pulse}}$$

If the combinational logic is too slow, the flip-flop may capture an incorrect value when the next clock pulse arrives.

This is why **propagation delay** is extremely important in sequential circuits.

The minimum allowable interval between clock pulses is influenced by the propagation delays of the combinational logic and other circuit components.

In other words:

$$\boxed{\text{Faster combinational logic} \Rightarrow \text{potentially faster clock}}$$

while excessive delay limits the maximum operating speed.

When Does a Flip-Flop Change State?

One of the most important characteristics of a synchronous sequential circuit is that a flip-flop does **not continuously change its stored value** whenever its input changes.

Instead, its state changes only when the appropriate **clock transition** occurs.

For example, a circuit may respond to the transition

$$0 \rightarrow 1$$

of the clock signal.

This is called a **rising-edge transition**.

When the clock is not active, the flip-flop maintains its stored value even if the combinational logic driving its input changes.

Therefore, the feedback path is effectively prevented from immediately changing the stored state.

The transition from one state to the next occurs only at the predetermined clock instants.

This gives us the fundamental sequential operation:

$$\boxed{\text{Present State} \xrightarrow{\text{clock pulse}} \text{Next State}}$$

Practice Exercise 5.1

Question

Describe the fundamental difference between the output of a combinational circuit and the output of a sequential circuit.

Answer

The output of a **combinational circuit** depends only on the inputs to the circuit.

$$\boxed{\text{Combinational output} = f(\text{inputs})}$$

The output of a **sequential circuit** depends on the inputs to the circuit **and the present state of the storage elements**.

$$\boxed{\text{Sequential output} = f(\text{inputs}, \text{present state})}$$

Therefore, the fundamental difference is the presence of **memory/state** in a sequential circuit.

2.11 STORAGE ELEMENTS: LATCHES

A **storage element** is a component in a digital circuit that can maintain a binary state for an indefinite amount of time, as long as power is supplied to the circuit. It keeps its stored value until an input signal tells it to change. The different storage elements mainly differ in **how many inputs they have** and **how those inputs control the stored binary state**.

There are two important categories of storage elements in this chapter: **latches** and **flip-flops**. A storage element that responds to **signal levels** is called a **latch**, whereas a storage element controlled by a **clock transition** is called a **flip-flop**.

Therefore:

Latch = level-sensitive

Flip-flop = edge-sensitive

Latches and flip-flops are closely related. In fact, **latches are the basic circuits from which flip-flops are constructed**. Although latches are useful for storing binary information and for asynchronous sequential circuits, they are generally not practical as the main storage elements in synchronous sequential circuits. Nevertheless, understanding latches is important because they form the foundation for understanding flip-flops.

SR Latch

The **SR latch** is one of the most basic memory circuits.

The letters mean:

S = Set

R = Reset

An SR latch can be constructed using either:

two cross-coupled NOR gates

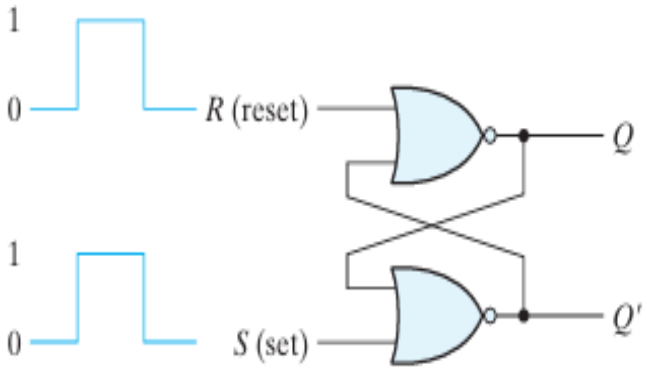
or

two cross-coupled NAND gates

"Cross-coupled" means that the output of each gate is connected back to an input of the other gate. This feedback is what allows the circuit to **remember its previous state**.

We will first understand the SR latch constructed using **NOR gates**.

SR NOR Latch



(a) Logic diagram

S	R	Q	Q′	
1	0	1	0	
0	0	1	0	(after $S = 1, R = 0$)
0	1	0	1	
0	0	0	1	(after $S = 0, R = 1$)
1	1	0	0	(forbidden)

(b) Function table

FIGURE 5.3
SR latch with NOR gates

The latch has two outputs:

$$Q$$

and

$$Q'$$

Normally, these two outputs are complements of each other.

There are two useful states.

Set State

When

$$Q = 1,\ Q' = 0$$

the latch is said to be in the **set state**.

Reset State

When

$$Q = 0,\ Q' = 1$$

the latch is said to be in the **reset state**.

Therefore,

$$\boxed{\text{Set state} \Rightarrow Q = 1}$$

$$\boxed{\text{Reset state} \Rightarrow Q = 0}$$

and normally

$$\boxed{Q' = Q'}$$

meaning $Q'$is the complement of Q.

Operation of the NOR SR Latch

For the NOR-based SR latch, the normal condition is

$$S = 0,\ R = 0$$

When both inputs are 0, the latch **holds its previous state**.

This is the memory property of the latch.

Suppose the latch is currently reset:

$$Q = 0,\ Q' = 1$$

If we momentarily apply

$$S = 1,\ R = 0$$

the latch changes to the set state:

$$Q = 1,\ Q' = 0$$

Afterward, if we remove the set signal and return $S$to 0, the latch **remains in the set state**.

So:

$$S = 1, R = 0 \Rightarrow Q = 1$$

and after returning to

$$S = 0, R = 0$$

the latch continues to remember

$$Q = 1$$

Similarly, to reset the latch, we momentarily apply

$$S = 0,\ R = 1$$

which causes

$$Q = 0,\ Q' = 1$$

When $R$is subsequently returned to 0, the latch remains in the reset state.

Thus, when

$$S = 0,\ R = 0$$

the latch can remain either in the set state or reset state depending on **which input was most recently activated**.

This is the actual memory mechanism.

Forbidden Condition of the NOR SR Latch

The most important problem with the NOR SR latch occurs when both inputs are simultaneously equal to 1:

$$\boxed{S = 1,\ R = 1}$$

In this condition, both outputs become

$$Q = 0$$

and

$$Q' = 0$$

But Q and Q' are supposed to be complements of each other. Having both outputs equal to 0 violates this requirement.

The real problem appears when both inputs are subsequently changed back to 0. The resulting state depends on **which input returns to 0 first**. Consequently, the resulting state cannot be predicted reliably.

The latch can therefore enter an **undefined, unpredictable, or metastable state**.

Hence, for the NOR SR latch:

$$\boxed{S = R = 1 \text{ is forbidden}}$$

This condition must be avoided in normal operation.

SR NOR Latch — Easy Function Table

The operation can be remembered as:

S	R	**Operation**
0	0	No change
1	0	Set $Q = 1$
0	1	Reset $Q = 0$

$S\ R$ Operation

1 1 Forbidden / indeterminate

The key point is that **0,0 means hold**, while the active signals are 1.

Why the Latch Has Memory

This is worth understanding rather than memorizing.

Suppose the latch is in the set state:

$$Q = 1,\ Q' = 0$$

Now both inputs are returned to

$$S = 0,\ R = 0$$

Nothing tells the circuit to change. Because of the cross-coupled feedback, the outputs maintain their previous values.

Therefore,

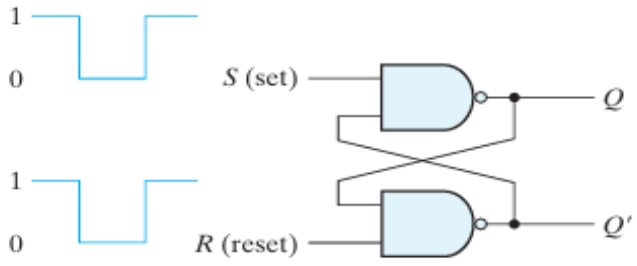
$$Q_{\text{next}} = Q_{\text{present}}$$

when

$$S = R = 0$$

This is how a latch stores one binary value.

SR Latch Using NAND Gates



(a) Logic diagram

S	R	Q	Q'	
1	0	0	1	
1	1	0	1	(after $S = 1, R = 0$)
0	1	1	0	
1	1	1	0	(after $S = 0, R = 1$)
0	0	1	1	(forbidden)

(b) Function table

FIGURE 5.4
SR latch with NAND gates

The SR latch can also be constructed using **two cross-coupled NAND gates**.

However, its input behavior is the complement of the NOR latch.

For the NAND latch, the inputs normally remain at:

$$S = 1,\ R = 1$$

A **0** signal is used to change the state.

To set the latch, apply

$$S = 0$$

which causes

$$Q = 1$$

and places the latch in the set state.

When *S*returns to 1, the latch remains in the set state.

To reset the latch, apply

$$R = 0$$

which causes

$$Q = 0$$

and places the latch in the reset state. When *R*returns to 1, the latch remains in the reset state.

Therefore, the NAND latch has the opposite active levels compared with the NOR latch.

For the NAND latch:

$$\boxed{0 = \text{active}}$$

For the NOR latch:

$$\boxed{1 = \text{active}}$$

Forbidden Condition of the NAND Latch

For the NAND latch, the forbidden condition is:

$$\boxed{S = 0,\ R = 0}$$

When both inputs are 0 simultaneously, the latch enters the undesirable condition.

Thus:

$$\boxed{\text{NOR latch forbidden condition: } S = R = 1}$$

while

NAND latch forbidden condition: $S = R = 0$

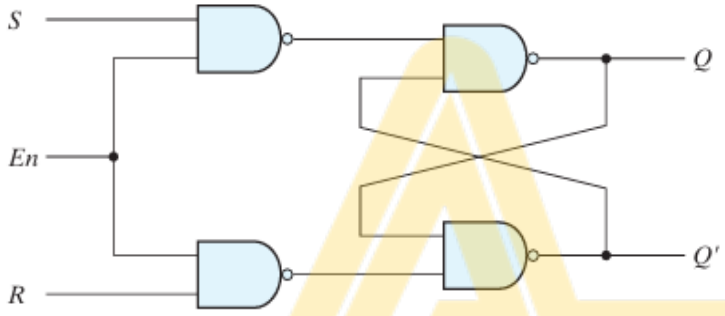
The NAND latch therefore requires the **complemented input signals** compared with the NOR latch. Because a 0 signal activates its inputs, it is sometimes called an

$S'R'$ latch

The primes indicate that the inputs are active in their complemented form.

SR Latch with Control Input

The operation of the basic SR latch can be modified by adding another input called the **enable input**.



(a) Logic diagram

En	*S*	*R*	Next state of *Q*
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$; reset state
1	1	0	$Q = 1$; set state
1	1	1	Indeterminate

(b) Function table

FIGURE 5.5
SR latch with control input

It is usually represented as

En

The purpose of $En$is to determine **when the S and R inputs are allowed to affect the latch**.

The controlled SR latch consists of the basic SR latch together with two additional NAND gates.

For this circuit, the enable input is **active-high**.

When

$$En = 0$$

the additional NAND gates prevent the S and R inputs from affecting the basic latch. The latch therefore remains in its current state.

So:

$En = 0 \Rightarrow$ No state change

When

$$En = 1$$

the S and R inputs are allowed to control the latch.

Controlled SR Latch Operation

When

$$En = 1,\ S = 1,\ R = 0$$

the latch enters the set state:

$$\boxed{Q = 1}$$

When

$$En = 1,\ S = 0,\ R = 1$$

the latch enters the reset state:

$$\boxed{Q = 0}$$

When

$$En = 1,\ S = 0,\ R = 0$$

there is **no change** in the stored state.

When

$$En = 0$$

the latch remains unchanged regardless of the values of $S$and R.

Thus, the enable input effectively acts as a **control switch** determining whether the latch is allowed to respond to S and R.

Indeterminate Condition in the Controlled SR Latch

There is still an undesirable condition.

When

$$S = 1,\ R = 1,\ En = 1$$

the additional NAND gates place 0's on both inputs of the basic SR latch. This is exactly the forbidden condition of the NAND-based SR latch.

Therefore, the state becomes indeterminate.

When the enable signal subsequently returns to 0, the resulting state depends on whether Sor Rgoes to 0 first.

Hence,

$$\boxed{S = R = En = 1}$$

is an indeterminate condition for this controlled SR latch.

Because this problem makes the circuit difficult to manage, this particular SR latch is seldom used in practice. Nevertheless, it remains extremely important because other useful latches and flip-flops are constructed from the SR latch.

Practice Exercise 5.2

(a) What input condition puts an SR NOR latch into an indeterminate state?

For an SR NOR latch:

$$\boxed{S = 1,\ R = 1}$$

So the answer is:

$$\boxed{\text{Both inputs are 1}}$$

(b) What input condition puts an SR NAND latch into an indeterminate state?

For an SR NAND latch:

$$\boxed{S = 0,\ R = 0}$$

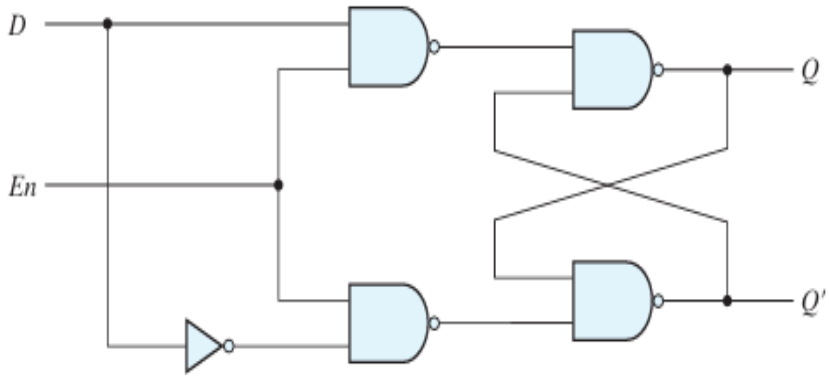
So the answer is:

$$\boxed{\text{Both inputs are 0}}$$

These answers are directly given in the textbook.

D Latch — Transparent Latch

The undesirable condition of the SR latch can be eliminated by ensuring that $S$and Rcan **never be 1 simultaneously**.



(a) Logic diagram

En	D	Next state of Q
0	X	No change
1	0	Q = 0; reset state
1	1	Q = 1; set state

(b) Function table

FIGURE 5.6
D latch

This is achieved using a **D latch**.

The D latch has only two inputs:

$$\boxed{D = \text{Data}}$$

and

$$\boxed{En = \text{Enable}}$$

The D input is connected directly to the S input, while its complement is connected to the R input:

$$S = D$$
$$R = D'$$

Therefore, if

$$D = 1$$

then

$$S = 1,\ R = 0$$

and the latch is set.

If

$$D = 0$$

then

$$S = 0,\ R = 1$$

and the latch is reset.

Since $D$and D'can never both be 1 at the same time, the forbidden SR condition is eliminated.

Operation of the D Latch

When

$$En = 0$$

the latch is disabled. The D input cannot change the stored state.

Therefore:

$$\boxed{En = 0 \Rightarrow Q_{\text{next}} = Q}$$

The previous value is retained.

When

$$En = 1$$

the D input is allowed to control the output.

If

$$D = 1$$

then

$$Q = 1$$

and if

$$D = 0$$

then

$$Q = 0$$

Therefore, while the enable input is active,

$$\boxed{Q = D}$$

This is the fundamental operation of the D latch.

Why Is It Called a Transparent Latch?

The D latch is also called a **transparent latch**.

This name comes from what happens when the enable input is asserted.

When

$$En = 1$$

the output follows the input:

$$\boxed{Q = D}$$

So if Dchanges from 0 to 1, Qchanges accordingly. If Dchanges from 1 to 0, Qalso follows it.

In other words, while the latch is enabled, there is effectively a path from the input $D$to the output Q.

The latch is therefore said to be **transparent** to the data input.

When the enable signal is de-asserted,

$$En = 0$$

the connection is effectively closed and the value that was present at $D$when the enable was de-asserted is retained at Q.

Thus:

$$\boxed{En = 1 \Rightarrow Q \text{ follows } D}$$

$$\boxed{En = 0 \Rightarrow Q \text{ holds its previous value}}$$

This is the most important thing to remember about a transparent latch.

D Latch Function Table

Enable En	D	**Next** Q
0	X	No change
1	0	$Q = 0$, reset
1	1	$Q = 1$, set

Here Xmeans **don't care**. When $En = 0$, the value of Ddoes not matter because the latch is disabled.

Active-High and Active-Low Enable

The enable signal can be designed to be either **active-high** or **active-low**.

In the circuit discussed here, the enable is active-high:

$$\boxed{En = 1 \Rightarrow \text{enabled}}$$

An inverter can be placed at the enable input so that the external control signal becomes active-low.

Then:

$$\boxed{En = 0 \Rightarrow \text{enabled}}$$

The actual choice depends on the physical circuit and how the external control signal is defined.

Latch Symbols

The textbook represents latches using **rectangular block symbols**.

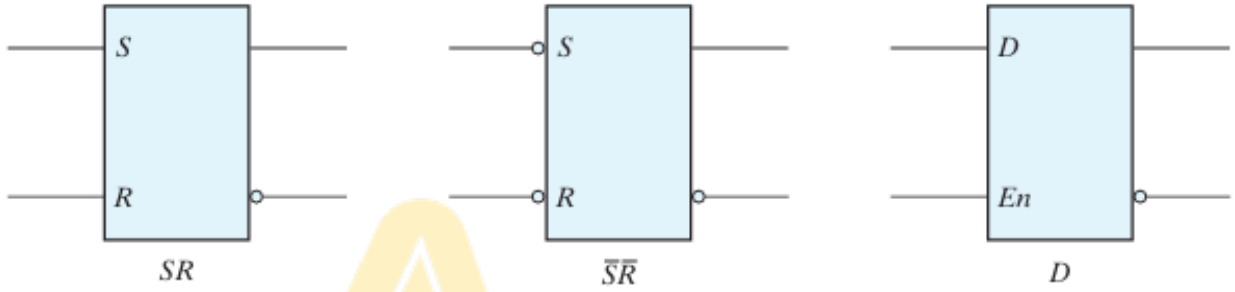


FIGURE 5.7
Graphic symbols for latches

Inputs are placed on the **left** side and outputs on the **right** side.

The normal output is represented by Q, while the complementary output is represented by Q'. A bubble indicates the complemented output.

For the SR latch, the inputs are labelled

$$S, R$$

For a NAND latch, bubbles are added to the input terminals to show that setting and resetting occur using logic 0 signals.

For a D latch, the inputs are labelled

$$D, En$$

These symbols provide a convenient way of representing the storage element without drawing the individual gates.

Practice Exercise 5.3

Question

Describe the functionality of a transparent latch.

Answer

A transparent latch has a **data input**, an **enable input**, and an **output**.

When the enable input is asserted, the output follows the input:

$$\boxed{En = 1 \Rightarrow Q = D}$$

When the enable input is de-asserted, the output retains the value that was present at the input when the enable signal was de-asserted:

$$\boxed{En = 0 \Rightarrow Q \text{ holds its previous value}}$$

Therefore, the simplest way to remember a transparent latch is:

$$\boxed{\text{Enable ON} \rightarrow \text{Output follows input}}$$

$$\boxed{\text{Enable OFF} \rightarrow \text{Output remembers}}$$

The textbook gives this exact functionality in Practice Exercise 5.3.

2.12 STORAGE ELEMENTS: FLIP-FLOPS

The previous section discussed **latches**, which are **level-sensitive** storage elements. The main problem with a latch in a synchronous sequential circuit is that it can keep responding to changes in its input for the entire time the clock is at its active level. A **flip-flop** is designed to solve this problem by changing its state only at a particular **transition, or edge, of the clock**.

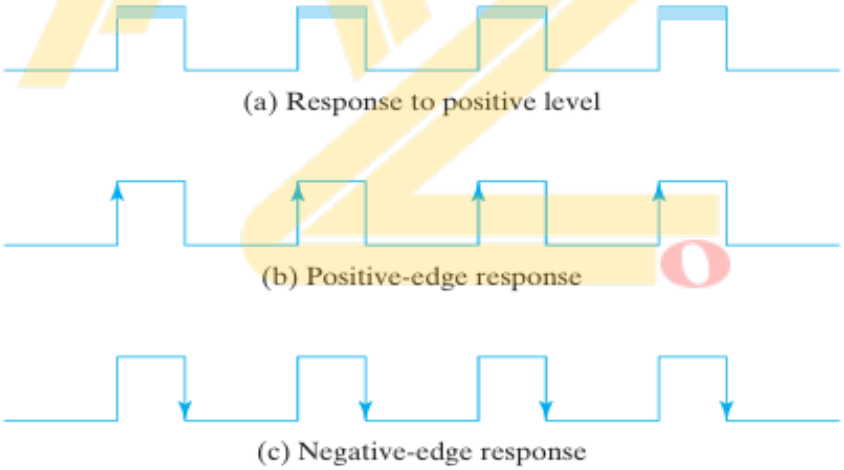


FIGURE 5.8
Clock response in latch and flip-flop

Why do we need flip-flops?

A change in the control input of a latch or flip-flop that causes its state to change is called a **trigger**. A D latch with pulses applied to its control input behaves somewhat like a flip-flop when the pulse goes to logic 1, but while the pulse remains at logic 1, any change in the data input can still change the output. Therefore, the latch remains sensitive during the whole active level of the clock.

This creates a serious problem in a sequential circuit containing feedback. The output of a flip-flop is fed back through combinational logic to the inputs of the storage elements. If latches are used, their state can begin changing as soon as the clock becomes active. The newly generated state can travel through the combinational circuit and reach the input of another latch while the same clock pulse is still active. The latch can then respond again, causing another state change. This process may continue for as long as the clock remains at its active level, making the final state unpredictable. Therefore, the output of one latch cannot safely be connected, directly or through combinational logic, to the input of another latch when all the latches are controlled by the same clock.

A flip-flop avoids this problem because it is designed to respond only to a **clock transition**, rather than to the entire clock level. A clock pulse has two transitions: when it changes from $0 \rightarrow 1$, called the **positive edge**, and when it changes from $1 \rightarrow 0$, called the **negative edge**. Thus, a flip-flop can be designed to respond either to the positive edge or to the negative edge of the clock.

There are two basic ways of converting the level-sensitive behavior of a latch into edge-triggered behavior. One method is to use **two latches in a special arrangement** so that the output is isolated from the input while the input is changing. The other method is to construct a circuit that responds only during a clock transition and remains disabled during the rest of the clock pulse.

Edge-Triggered D Flip-Flop

One important type of flip-flop is the **edge-triggered D flip-flop**. It can be constructed using **two D latches and an inverter**. This arrangement is commonly called a **master–slave flip-flop**. The first latch is called the **master**, and the second latch is called the **slave**.

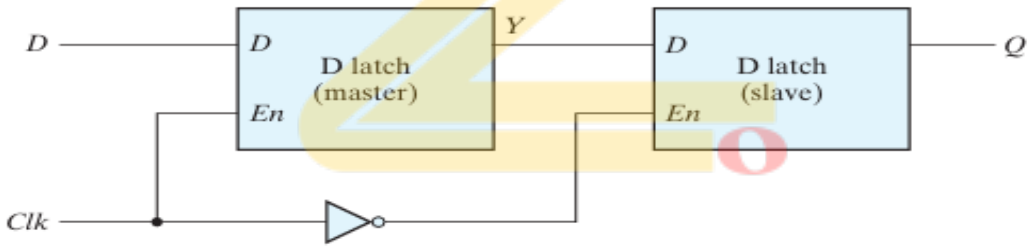


FIGURE 5.9
Master–slave *D* flip-flop

Consider the negative-edge-triggered master–slave D flip-flop. When the clock is 0, the inverter output is 1. Therefore, the slave latch is enabled and its output $Q$is equal to the master output Y. At the same time, the master latch is disabled because Clk $= 0$.

When the clock changes from $0 \rightarrow 1$, the data present at the external Dinput is transferred into the master latch. However, the slave is disabled while the clock remains at 1. Therefore, even if Dchanges

during this interval, those changes can affect the master output Y, but they cannot affect the final output Q.

When the clock returns from $1 \rightarrow 0$, the master becomes disabled and is isolated from the Dinput. At the same time, the slave becomes enabled, so the value stored at $Y$is transferred to Q. Consequently, the output changes only during the **negative transition of the clock**.

The important behavior can therefore be understood as follows:

D is sampled by the master, and Q changes at the negative edge of the clock.

The master–slave flip-flop has three important properties. Its output can change only once for each clock cycle, the change is triggered by the negative edge of the clock, and the value appearing at the output is the value that was stored in the master immediately before the negative edge occurred.

It is also possible to construct a flip-flop that changes its output on the **positive edge** of the clock. This can be achieved by adding another inverter in the clock path. In this arrangement, the master and slave operate in the opposite sequence, causing the output to change when the clock makes its $0 \rightarrow 1$transition.

Another Edge-Triggered D Flip-Flop Construction

Another implementation of an edge-triggered D flip-flop uses **three SR latches**. Two of the latches respond to the external $D$and clock inputs, while the third latch provides the output of the flip-flop.

When the clock is 0, the $S$and Rinputs of the output latch are maintained at logic 1. Therefore, the output remains in its existing state.

When the clock changes to 1, the value of Ddetermines what happens. If $D = 0$, the reset input Rbecomes 0, causing the flip-flop to reset:

$$Q = 0$$

Once the flip-flop is reset, further changes in Dwhile the clock remains at 1do not affect the output. Similarly, if $D = 1$when the clock changes from $0 \rightarrow 1$, the set input Sbecomes 0, causing the flip-flop to enter the set state:

$$Q = 1$$

Again, changes in Dwhile the clock remains at 1do not affect the output.

This gives the fundamental behavior of a positive-edge-triggered D flip-flop:

At the positive edge, Q takes the value of D

Positive-Edge and Negative-Edge Flip-Flops

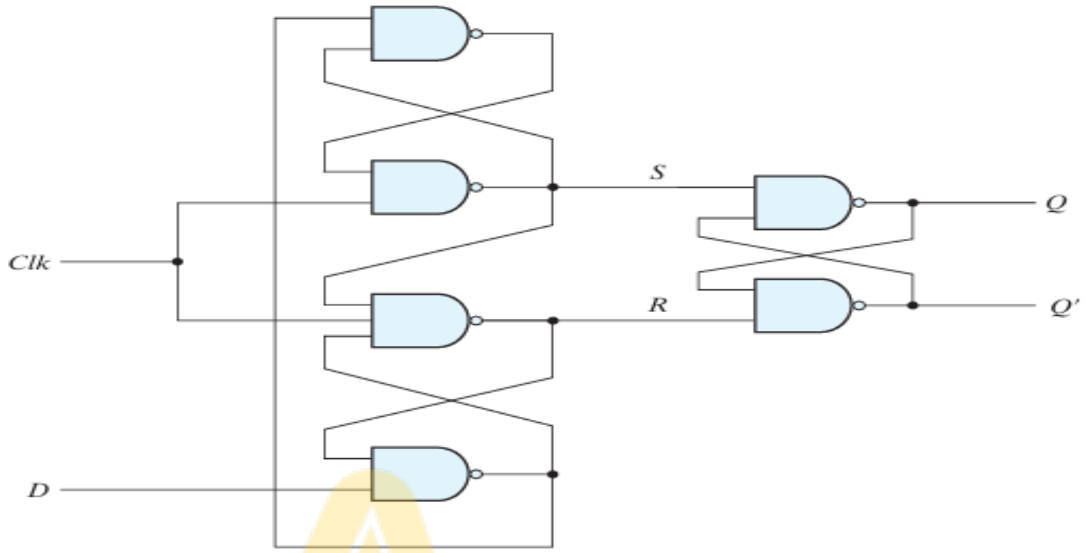


FIGURE 5.10
D-type positive-edge-triggered flip-flop

For a **positive-edge-triggered D flip-flop**, the value of $D$is transferred to $Q$when the clock changes from $0 \rightarrow 1$.

$$\boxed{0 \rightarrow 1:\ Q(t+1) = D}$$

After this transition, changes in Ddo not affect Quntil the next positive clock edge. A negative clock transition $1 \rightarrow 0$also does not affect the output. Therefore, the flip-flop responds specifically to the rising edge and not to the steady clock levels.

For a **negative-edge-triggered D flip-flop**, the corresponding triggering transition is:

$$1 \rightarrow 0$$

So the output changes when the clock makes its falling transition.



FIGURE 5.11
Graphic symbol for edge-triggered *D* flip-flop

In the circuit symbol, an **arrowhead-like dynamic indicator** at the clock input indicates that the device is edge-triggered. If there is **no bubble** associated with the dynamic clock input, it represents a positive-edge-triggered flip-flop. A **bubble** indicates a negative-edge-triggered flip-flop.

Practice Exercise 5.4

A **positive-edge flip-flop** is a flip-flop that is activated by the **rising, or positive, edge of the clock signal**.

Setup Time, Hold Time, and Propagation Delay

When using edge-triggered flip-flops, the timing relationship between the input data and the clock is extremely important.

The **setup time** is the minimum amount of time for which the Dinput must remain stable **before** the active clock transition occurs.

The **hold time** is the minimum amount of time for which the Dinput must remain stable **after** the active clock transition occurs.

Therefore, for reliable operation, the data should not change too close to the triggering clock edge.

There is also a **propagation delay** associated with the flip-flop. It is the time interval between the triggering clock edge and the time at which the output becomes stable at its new state. These timing parameters are specified by manufacturers for particular logic families.

In simple words:

Setup time = data must be stable before the clock edge

Hold time = data must remain stable after the clock edge

Propagation delay = time taken for Q to respond after the clock edge

Other Flip-Flops

The **D flip-flop** is particularly economical because it requires a relatively small number of gates. Other flip-flops can be constructed using a D flip-flop together with external logic. Two other important types are the **JK flip-flop** and the **T flip-flop**.

A flip-flop can essentially perform three operations:

Set $\rightarrow Q = 1$

Reset $\rightarrow Q = 0$

Complement $\rightarrow Q = Q'$

The D flip-flop can set or reset its output depending on the value of Dimmediately before the clock transition. The JK flip-flop has two inputs and can perform all three operations: set, reset, and complement.

JK Flip-Flop

The two inputs of a JK flip-flop are called $J$and K. The Jinput is associated with **setting** the flip-flop, while the Kinput is associated with **resetting** it.

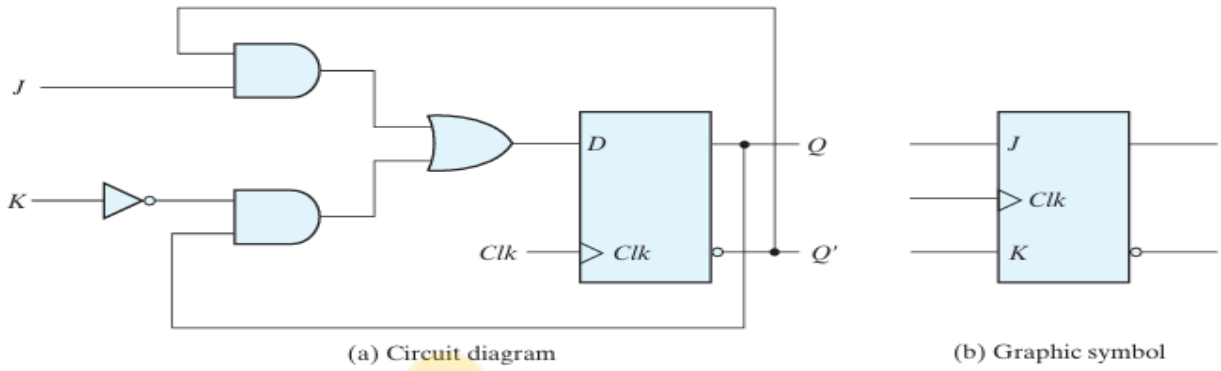


FIGURE 5.12
JK flip-flop

The JK flip-flop can be constructed using a D flip-flop and additional gates. The input applied to the D terminal is:

$$\boxed{D = JQ' + K'Q}$$

This equation allows us to understand all four possible combinations of $J$and K.

When $J = 1, K = 0$

Substituting into the equation:

$$D = (1)Q' + (0)'Q$$
$$D = Q' + Q$$

Since

$$Q' + Q = 1$$

we get:

$$D = 1$$

Therefore, at the next clock edge:

$$\boxed{Q(t+1) = 1}$$

So the flip-flop is **set**.

When $J = 0, K = 1$

$$D = (0)Q' + (1)'Q$$
$$D = 0 + 0Q$$
$$D = 0$$

Therefore:

$$\boxed{Q(t+1) = 0}$$

So the flip-flop is **reset**.

When $J = 1, K = 1$

$$D = (1)Q' + (1)'Q$$
$$D = Q' + 0$$
$$D = Q'$$

Therefore, at the next clock edge:

$$\boxed{Q(t+1) = Q'(t)}$$

The output is **complemented**.

When $J = 0, K = 0$

$$D = (0)Q' + (0)'Q$$
$$D = Q$$

Therefore:

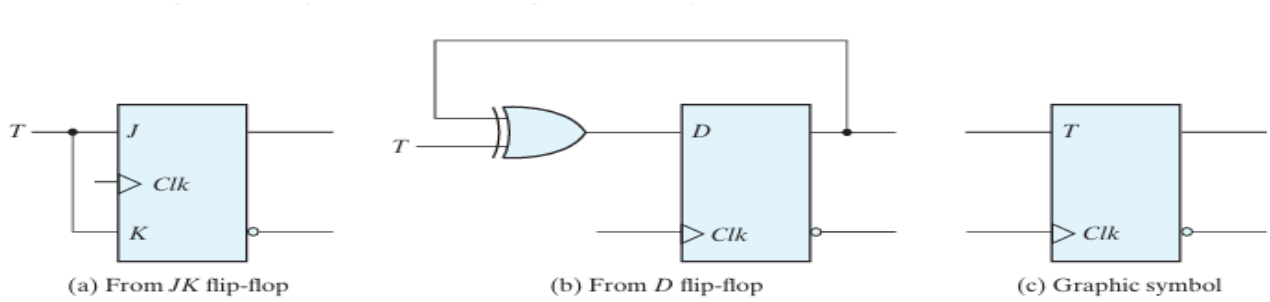
$$\boxed{Q(t+1) = Q(t)}$$

The output remains unchanged.

Thus, the JK flip-flop provides four possibilities: **no change, reset, set, and complement**.

T Flip-Flop

The **T flip-flop**, where T stands for **toggle**, is a complementing flip-flop. It can be obtained from a JK flip-flop by connecting the *J* and *K* inputs together:



(a) From *JK* flip-flop (b) From *D* flip-flop (c) Graphic symbol

FIGURE 5.13
T flip-flop

$$J = K = T$$

When

$$T = 0$$

we have

$$J = K = 0$$

and the clock edge does not change the output.

When

$$T = 1$$

we have

$$J = K = 1$$

and the clock edge complements the output. Therefore:

$$\boxed{T = 0 \Rightarrow Q(t+1) = Q(t)}$$
$$\boxed{T = 1 \Rightarrow Q(t+1) = Q'(t)}$$

Because it changes its state from $0 \rightarrow 1 \rightarrow 0 \rightarrow 1$, the T flip-flop is particularly useful in designing **binary counters**.

A T flip-flop can also be constructed from a D flip-flop and an XOR gate. The input to the D flip-flop is:

$$\boxed{D = T \oplus Q}$$

Using the XOR expansion:

$$\boxed{D = TQ' + T'Q}$$

When $T = 0$,

$$D = Q$$

so there is no change.

When $T = 1$,

$$D = Q'$$

so the output complements.

Characteristic Tables

A **characteristic table** describes the logical behavior of a flip-flop. It gives the **next state** as a function of the flip-flop's **inputs and present state**.

The notation

$$Q(t)$$

represents the **present state**, meaning the state immediately before the clock edge.

The notation

$$Q(t+1)$$

represents the **next state**, meaning the state resulting from the clock transition.

The clock edge itself is not explicitly included in the characteristic table because its occurrence is understood to take place between $t$and $t+1$.

For the **JK flip-flop**, the four cases are:

$$J = 0, K = 0 \Rightarrow Q(t+1) = Q(t)$$
$$J = 0, K = 1 \Rightarrow Q(t+1) = 0$$
$$J = 1, K = 0 \Rightarrow Q(t+1) = 1$$
$$J = 1, K = 1 \Rightarrow Q(t+1) = Q'(t)$$

These correspond respectively to **no change, reset, set, and complement**.

For the **D flip-flop**, the next state depends only on D:

$$\boxed{Q(t+1) = D}$$

Therefore, if $D = 0$, the next state is 0, and if $D = 1$, the next state is 1. The present value of Qdoes not matter.

For the **T flip-flop**:

$$T = 0 \Rightarrow Q(t+1) = Q(t)$$

$$T = 1 \Rightarrow Q(t+1) = Q'(t)$$

Thus, $T = 0$means **no change**, while $T = 1$means **complement**.

Characteristic Equations

The logical behavior represented by a characteristic table can also be written algebraically as a **characteristic equation**.

For the D flip-flop:

$$\boxed{Q(t+1) = D}$$

This means that after the active clock transition, the next state is simply the value of D.

For the JK flip-flop:

$$\boxed{Q(t+1) = JQ' + K'Q}$$

Here Qrepresents the output value immediately before the clock edge.

For the T flip-flop:

$$\boxed{Q(t+1) = T \oplus Q}$$

or, using the Boolean expression for XOR,

$$\boxed{Q(t+1) = TQ' + T'Q}$$

These three equations are **extremely important for sequential-circuit analysis and design**:

$$\boxed{\text{D: } Q^+ = D}$$
$$\boxed{\text{JK: } Q^+ = JQ' + K'Q}$$
$$\boxed{\text{T: } Q^+ = T \oplus Q}$$

where $Q^+$is another common notation for $Q(t+1)$.

Direct Inputs

Some flip-flops have additional **asynchronous inputs** that can force the flip-flop into a particular state without waiting for the clock.

The input that forces the flip-flop to 1is called **preset** or **direct set**.

The input that forces the flip-flop to 0is called **clear** or **direct reset**.

When a digital system is powered on, the initial states of its flip-flops may be unknown. Direct inputs are therefore useful for placing all the flip-flops into a known initial state before normal clocked operation begins.

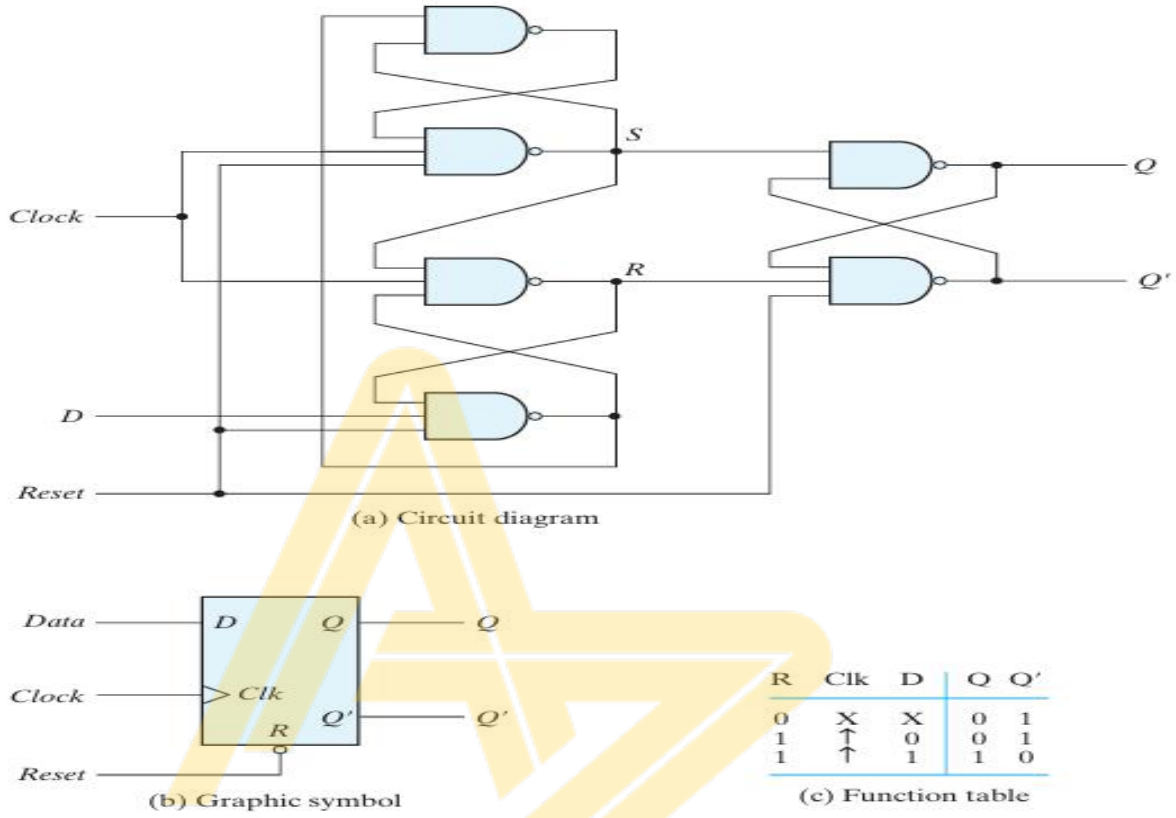


FIGURE 5.14
D flip-flop with asynchronous reset

For example, consider a positive-edge-triggered D flip-flop having an **active-low asynchronous reset**.

The reset input is active when:

$$R = 0$$

When $R = 0$, the flip-flop is immediately reset:

$$\boxed{Q = 0}$$

This happens independently of the values of $D$and the clock. Normal clock operation resumes only after the reset returns to:

$$R = 1$$

Then, on every positive edge of the clock, the value of $D$is transferred to Q.

The bubble shown on the reset input of the circuit symbol indicates that the reset is **active-low**. A direct set input is generally marked with S.

Practice Exercise 5.5 — Functionality of a D-Type Flip-Flop

A **D-type flip-flop** has a Ddata input and a clock input, and it may additionally have asynchronous or synchronous set/reset inputs.

When set or reset signals are not asserted, the clock controls the transfer of $D$to Q. If the set or reset inputs are **asynchronous**, they operate independently of the clock: set forces

$$Q = 1$$

and reset forces

$$Q = 0.$$

If the set or reset inputs are **synchronous**, they take effect at the active clock edge.

The main idea to remember

The easiest way to distinguish the storage elements discussed here is:

Latch: responds to a **clock/control level**.

Flip-flop: responds to a **clock edge**.

For an edge-triggered D flip-flop:

$$\boxed{\text{Positive edge: } 0 \rightarrow 1}$$

or

$$\boxed{\text{Negative edge: } 1 \rightarrow 0}$$

depending on its design.

The three important flip-flop characteristic equations are:

$$\boxed{D \text{ flip-flop: } Q(t+1) = D}$$
$$\boxed{JK \text{ flip-flop: } Q(t+1) = JQ' + K'Q}$$
$$\boxed{T \text{ flip-flop: } Q(t+1) = T \oplus Q}$$

And the essential operations are:

$$\boxed{\text{Set} \rightarrow Q = 1}$$
$$\boxed{\text{Reset} \rightarrow Q = 0}$$
$$\boxed{\text{Toggle/Complement} \rightarrow Q = Q'}$$

The most important conceptual point is that **a latch is level-sensitive, whereas a flip-flop is edge-sensitive**. This edge-triggered behavior is what makes flip-flops suitable as the principal storage elements in synchronous sequential circuits.

Thank You

We're glad to be part of your engineering journey.
Keep exploring, keep innovating, and keep growing.

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

Quality notes, simplified explanations, and student-focused resources for every semester.

Connect With Us:

www.azdocuments.in [Click Here]

@azdocuments [Click Here]

Join Our Student Community: [Click Here]

© 2025 AZ Documents. All rights reserved.

This material is intended for educational purposes only.
Redistribution without permission is prohibited.